



**Universidad
Europea** VALENCIA

GRADO EN INGENIERÍA DE ORGANIZACIÓN INDUSTRIAL

Integración de reconocimiento de voz para la automatización de registros en un sistema ERP

Presentado por:

BYRON ANDRES PEREZ CHAVEZ

Dirigido por:

JOSE MARIA FERNANDEZ NAVARRO

CURSO ACADÉMICO 2025-2026

AGRADECIMIENTOS

A todos los que han formado parte de este camino: por la paciencia, por las pruebas, por las ideas a deshora y por confiar en que un audio de Telegram podía convertirse en un Trabajo Fin de Grado. Nada de esto habría sido posible solo.

RESUMEN

La ineficiencia en el registro manual de datos en sistemas ERP limita la productividad operativa en entornos de campo, donde el acceso a un ordenador no siempre es viable. Para dar respuesta a esta necesidad, se desarrolla una interfaz conversacional inteligente que automatiza dichos registros en Odoo mediante comandos de voz a través de Telegram. El sistema emplea Google Gemini para el procesamiento del lenguaje natural y la transcripción multimodal, interpretando instrucciones complejas y sincronizando entidades en tiempo real con la API XML-RPC de Odoo. La solución se conteneriza y despliega en Google Cloud Run bajo un modelo serverless, validándose en cuatro entornos contrastados: proceso manual, bot en PC local y bot en Cloud Run con CPU throttled y siempre activa. La precisión de transcripción alcanzó el 87,5 % (WAR). La configuración optimizada reduce el tiempo de creación de registros en un 50,8% respecto al método manual (26,15s frente a 53,17s), igualando el rendimiento local. Ergonómicamente, el tiempo activo del operario se reduce entre un 58,1 % y un 62,4 % al desacoplar grabación y procesamiento. Los resultados confirman la viabilidad técnica de integrar IA conversacional en sistemas de gestión empresarial como herramienta real de transformación digital.

Palabras clave: *Odoo, Automatización, IA, Gemini, ERP.*

ABSTRACT

The inefficiency in manual data entry within ERP systems limits operational productivity in field environments, where access to a computer is not always viable. To address this need, an intelligent conversational interface is developed to automate these records in Odoo using voice commands through Telegram. The system employs Google Gemini for natural language processing and multimodal transcription, interpreting complex instructions and synchronizing entities in real time with Odoo's XML-RPC API. The solution is containerized and deployed on Google Cloud Run under a serverless model, validated across four contrasted environments: manual process, local PC bot, and Cloud Run bot with both throttled and always-on CPU. Transcription accuracy reached 87.5% (WAR). The optimized configuration reduces record creation time by 50.8% compared to the manual method (26.15s versus 53.17s), matching local performance. Ergonomically, the operator's active time is reduced by 58.1% to 62.4% by decoupling recording and processing. The results confirm the technical feasibility of integrating conversational AI into business management systems as a real tool for digital transformation.

Key words: Odoo, Automation, AI, Gemini, ERP.

ÍNDICE DE CONTENIDOS

Glosario.....	9
Capítulo 1. Introducción.....	11
1.1. Contexto.....	11
1.2. Planteamiento del Problema	12
1.3. Justificación	13
1.4. Alcance: Qué Hace el Bot y Qué Queda Fuera.....	15
Capítulo 2. Objetivos del Proyecto	18
2.1. Objetivo General.....	18
2.2. Objetivos Específicos.....	18
Capítulo 3. Marco Teórico y Tecnológico	20
3.1. Reconocimiento de Voz (STT): Desafíos en entornos industriales	20
3.2. Chatbots e Interfaces Conversacionales: Evolución	21
3.3. IA Generativa (LLMs): Google Gemini y su capacidad de comprensión	22
3.4. Protocolos: XML-RPC vs REST en integraciones	24
3.4.1. XML-RPC.....	24
3.4.2. REST.....	24
3.4.3. Comparativa en Integraciones	25
3.5. Sistemas ERP y Odoo: Arquitectura Modular	26
3.6. Casos de Éxito y Adopción en la Industria: Asistentes de Voz en ERPs	27
3.7. Métricas de Evaluación en Ingeniería de Voz y Reconocimiento de Habla (STT)	28
Capítulo 4. Metodología y Planificación del Proyecto	29
4.1. Metodología de Desarrollo: Enfoque Ágil y Scrum	29
4.2. Planificación de Desarrollo (Personal Scrum)	29
4.3. Diseño Experimental y Estrategia de Validación (Proof of Concept).	31
4.4. Recursos del Proyecto.....	32
4.4.1. Recursos de Hardware e Infraestructura	32
4.4.2. Recursos de Software	32
4.5. Viabilidad del Proyecto.....	34
4.5.1. Viabilidad Económica	34
4.5.2. Viabilidad Técnica	36
Capítulo 5. Desarrollo de la solución.....	38
5.1. Arquitectura del Sistema y <i>Flujo de Datos</i>	38
5.2. Configuración: Variables de Entorno (.env)	39
5.3. Conexión con Odoo (Clase OdooConnector)	41

5.4 Inteligencia Artificial y Procesamiento de Lenguaje.....	43
5.5. Lógica del Bot y Gestión de Estado.....	44
5.6 Integración de Voz (STT/TTS).....	45
5.7 Despliegue en la Nube: Google Cloud Run y Docker.	46
Capítulo 6. Pruebas y Validación del Sistema	47
6.1. Entorno y Metodología de Validación.....	47
6.2 Validación de Mercado: Visita a Mercafilo y AHORA Soluciones	48
6.3. Panel de Validación: Estudiantes y Proyección Profesional.....	49
6.4. Escenario de Prueba 1: Creación de Registros desde Audio.....	50
6.5. Escenario de Prueba 2: Búsqueda y Consulta de Información	51
6.6. Validación Técnica: Precisión de la Transcripción de Gemini.....	52
6.7. Análisis de Eficiencia Empírica: Comparativa de los Cuatro Entornos	55
6.7.1. Benchmark del Proceso Manual	55
6.7.2. Matriz Comparativa General: Cuatro Entornos y Tres Módulos	56
6.7.3. Desglose Módulo Contactos: Análisis Fase a Fase	57
6.7.4. Desglose Módulo CRM: Análisis Fase a Fase.....	58
6.7.5. Desglose Módulo Consultas: Análisis Fase a Fase	60
6.7.6. Causa Raíz de la Penalización por Throttling: Diagnóstico Técnico	61
6.7.7. Desglose del Rendimiento del Panel Empírico por Grupos de Validación	61
6.8. Análisis de Ergonomía Cognitiva: El Coste Real para el Operario	63
6.9. Análisis de Robustez Operacional: Tipificación de Errores y Límites del Sistema.....	64
6.10. Síntesis y Valoración Global de la Validación	66
Capítulo 7. Objetivos de Desarrollo Sostenible (ODS)	67
7.1. ODS 9: Industria, Innovación e Infraestructuras	67
7.2. ODS 8: Trabajo Decente y Crecimiento Económico (Mejora Productividad del Empleado)	68
7.3. ODS 12: Producción y Consumo Responsables (Eficiencia Operativa).....	70
Capítulo 8. Conclusiones, Limitaciones y Líneas Futuras.....	71
8.1. Conclusiones.....	71
8.2. Limitaciones.....	72
8.3. Líneas Futuras.....	73
REFERENCIAS	75
ANEXO I	78
ANEXO II	79

ÍNDICE DE FIGURAS

Ilustración 1 Evolución de los Sistemas ERP en la Era Digital	11
Ilustración 2. Diagrama de Alcance del Bot de Odoo	17
Ilustración 3 Comparativa Conceptual de XML-RPC y REST	25
Ilustración 4 Arquitectura Modular Típica de un sistema ERP como Odoo	26
Ilustración 5 Iteraciones Ágiles del TFG	29
Ilustración 6 Arquitectura General del Sistema (Telegram-IA-Odoo).....	39
Ilustración 7 Flujo de Conexión y Autenticación con Odoo mediante XML-RPC.....	42
Ilustración 8 Diagrama de Flujo de Procesamiento IA para Extracción de Datos	44
Ilustración 9 Escenario: Creación de Oportunidad desde Nota de Voz	51
Ilustración 10 Flujo de Búsqueda de Información mediante Voz	52
Ilustración 11 Ejemplo de Transcripción de Voz a Texto con Gemini	54
Ilustración 12 Interfaz de Usuario Final en Telegram.....	54
Ilustración 13 Esquema de Digitalización y Mejora de Infraestructura	68

ÍNDICE DE TABLAS

Tabla 1 Comparativa: Entrada Manual vs. Entrada por Voz e IA.....	14
Tabla 2 Comparativa Técnica de Modelos LLM para Integración ERP	23
Tabla 3 Presupuesto Real Ejecutado (Dic 2025 - Mar 2026).....	35
Tabla 4 Variables de Entorno Esenciales del Sistema	40
Tabla 5 Composición de los Paneles de Validación y Proyección.....	50
Tabla 6 Comparativa de Precisión STT	53
Tabla 7 Benchmark Operativo: Resumen Estadístico del Proceso Manual (n=160)	55
Tabla 8 Matriz Comparativa de Latencia Media: Manual vs. Bot (Cuatro Entornos)	56
Tabla 9 Desglose por Fases — Módulo Contactos (Tiempos Medios en Segundos).....	57
Tabla 10 Desglose por Fases — Módulo CRM (Tiempos Medios en Segundos).....	59
Tabla 11 Desglose por Fases — Módulo Consultas (Tiempos Medios en Segundos).....	60
Tabla 11a Desglose Estadístico de Rendimiento por Grupos de Validación (n=160)	62
Tabla 12 Análisis de Ergonomía Cognitiva: Tiempo Activo vs. Tiempo de Proceso.....	63
Tabla 13 Proyección de impacto organizativo (Ahorro de tiempo Activo)	64
Tabla 14 Catálogo de Errores del Sistema: Tipificación, Frecuencia y Mitigación.....	65
Tabla 15 . Contribuciones del Proyecto al ODS 8	69
Tabla 16 Matriz de Cumplimiento de Objetivos del Proyecto	72

Glosario de términos y acrónimos

Para facilitar la lectura y comprensión técnica de este documento, se definen a continuación los términos empleados en los ámbitos de la ingeniería de software, telecomunicaciones, inteligencia artificial y organización industrial:

Glosario I

Término / Acrónimo	Definición en el contexto del presente proyecto
API (<i>Application Programming Interface</i>)	Interfaz de programación que permite la comunicación entre sistemas independientes. El proyecto se articula conectando tres: la API de Telegram, la API de IA (Gemini) y la API externa del ERP (Odoo).
Cloud Run / Serverless	Paradigma de computación en la nube de Google donde no se gestionan servidores físicos. El código (contenedor) se ejecuta bajo demanda, facturando recursos únicamente durante los milisegundos de procesamiento activo.
Cold Start (<i>Arranque en frío</i>)	Penalización de latencia masiva (hasta 30-40 segundos) que ocurre en infraestructura <i>serverless</i> cuando el sistema lleva tiempo inactivo y debe arrancar un contenedor desde cero para procesar una nueva petición de voz.
Docker	Tecnología de contenedorización que empaqueta el código del bot de Python junto con sus dependencias del sistema operativo (como la librería FFmpeg), garantizando que se ejecute idénticamente en un PC local o en la nube.
HitL (<i>Human-in-the-Loop</i>)	Patrón de diseño de sistemas de Inteligencia Artificial donde un humano mantiene el control final. En este TFG, se refleja en los botones de "Confirmar/Cancelar", evitando que la IA escriba en el ERP sin supervisión.
LLM (<i>Large Language Model</i>)	Modelo algorítmico de redes neuronales masivas. En este proyecto se utiliza Google Gemini (2.5 Flash), no solo para conversar, sino como motor de razonamiento para extraer datos no estructurados y convertirlos en esquemas válidos para el ERP.
OGG / WAV / PCM	Formatos de la señal de audio. Telegram captura la voz en OGG (comprimido). El código del sistema lo lee y manipula a nivel de bytes (inyectando cabeceras WAV) para extraer el audio crudo PCM requerido por la API nativa de Gemini.
ORM (<i>Object-Relational Mapping</i>)	Capa interna de Odoo que traduce código estructurado (Python) en consultas a la base de datos SQL. El bot no interactúa con la base de datos directamente, sino que envía comandos al ORM.
Prompt	Instrucción programada de contexto que se inyecta de forma invisible en cada petición enviada al modelo Gemini. Define el formato de salida JSON y las reglas matemáticas (esquema dinámico) que la IA debe respetar al extraer datos.

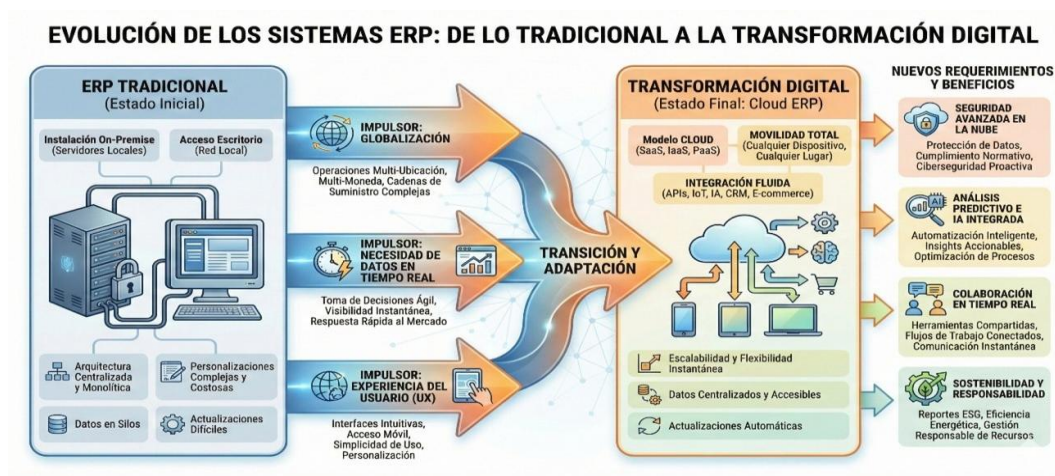
RTT (<i>Round-Trip Time</i>)	Tiempo físico (en milisegundos) que tarda un paquete de datos en viajar por internet desde el bot hasta los servidores de Google/Telegram y volver. Es la causa raíz que explica la latencia en el módulo de consultas del sistema.
STT (<i>Speech-to-Text</i>)	Transcripción de señales acústicas a texto plano. Este proyecto innova al no utilizar un motor STT secundario, delegando esta función de forma nativa a la capacidad multimodal de la API de Gemini.
Throttling (<i>CPU Throttled</i>)	Restricción técnica donde el servidor estrangula (congela) el uso del procesador si no hay tráfico de red activo. En el TFG se demuestra que esta configuración paraliza los procesos asíncronos de la IA en segundo plano.
Token	Unidad de medida de facturación de los LLM (1 token $\approx$ 3/4 de palabra). El presupuesto total del TFG (1,05 € de IA) se calcula en base a los millones de tokens de audio y texto consumidos en la validación.
WAR / WER	<i>Word Accuracy Rate</i> y <i>Word Error Rate</i> . Las dos métricas de ingeniería estándar empleadas para medir matemáticamente el éxito (WAR, 87,5%) o el fracaso (WER, 12,5%) de la IA al escuchar y transcribir al usuario.
Webhook	Método de comunicación pasiva donde Telegram avisa "en tiempo real" al servidor de que ha llegado un audio, evitando que el bot consuma recursos preguntando constantemente si hay mensajes nuevos.
XML-RPC	Protocolo de red estructurado en formato XML que sirve de "puerta de entrada" estándar a Odoo. Se emplea en el TFG porque permite ejecutar operaciones de creación y lectura en la base de datos de manera robusta y unificada.

Capítulo 1. Introducción

1.1. Contexto: La Transformación Digital y la Necesidad de Movilidad en Sistemas ERP

Los sistemas de planificación de recursos empresariales (ERP) se han consolidado durante las últimas décadas como el eje vertebrador de la gestión corporativa: centralizan la información, estandarizan los procesos y permiten tomar decisiones con datos actualizados. Hammer y Champy (1993), en su seminal obra sobre reingeniería de procesos, ya anticipaban que la integración sistémica de la información es el núcleo indispensable para la renovación operativa del negocio. La predicción fue correcta. Sin embargo, persiste una brecha entre lo que el ERP promete y lo que el personal de campo experimenta cuando intenta usarlo fuera de la oficina. El problema no es el sistema en sí, sino el contexto de uso. Los equipos de ventas, técnicos y personal de servicio trabajan en movimiento, y la irrupción de la Industria 4.0 (Lasi et al., 2014) con sus entornos conectados y sus procesos integrados digitalmente ha cambiado dónde y cuándo se genera la información que necesita entrar al ERP. Pero la interfaz de usuario de esos sistemas no ha evolucionado al mismo ritmo: sigue siendo, en esencia, un formulario de escritorio pensado para usarse con teclado, ratón y tiempo. La integración de tecnologías móviles y la optimización de la experiencia del usuario fuera de un entorno de oficina son, por tanto, desafíos clave. La adaptación de los sistemas ERP para permitir una interacción fluida en dispositivos móviles no solo mejora la eficiencia del personal de campo, sino que también facilita la captura de datos en tiempo real, lo que a su vez proporciona una visión más precisa y actualizada del estado de las operaciones empresariales.

Ilustración 1 Evolución de los Sistemas ERP en la Era Digital



Nota: Elaboración propia asistida por Google Gemini.

La Ilustración 1 resume de forma visual la brecha que justifica este proyecto. A la izquierda aparece el modelo ERP tradicional: servidor on-premise, acceso desde escritorio, datos que solo se actualizan cuando el empleado está en la oficina. A la derecha, el escenario que representa la transformación digital: arquitectura en la nube, acceso desde cualquier dispositivo y, sobre todo, la posibilidad de interactuar con el sistema en tiempo real desde el campo. El bot desarrollado en este TFG es, precisamente, la pieza que conecta esos dos mundos: permite que el personal de campo actualice Odoo con la misma naturalidad con la que envía una nota de voz por una aplicación de mensajería, sin necesidad de estar delante de un ordenador.

1.2. Planteamiento del Problema: La Ineficiencia de la Introducción Manual de Datos en Odoo por Parte de Comerciales/Técnicos de Campo

A pesar de los avances en la digitalización, una problemática persistente en muchas organizaciones es la ineficiencia asociada a la introducción manual de datos en sistemas ERP como Odoo, especialmente por parte del personal de campo. Comerciales y técnicos, que pasan una parte significativa de su jornada laboral fuera de la oficina, se enfrentan a la tarea de registrar visitas, actualizar estados de oportunidades, crear pedidos, reportar incidencias o documentar el progreso de tareas.

Este proceso manual de entrada de datos, que a menudo se realiza al final del día o en momentos de inactividad, es propenso a diversas ineficiencias:

- **Pérdida de Tiempo:** La navegación a través de interfaces de escritorio complejas en dispositivos móviles pequeños, o la espera hasta regresar a la oficina, consume un tiempo valioso que podría dedicarse a actividades de mayor valor añadido, como interactuar con clientes o realizar tareas técnicas.
- **Errores y Omisiones:** La introducción manual de datos bajo presión de tiempo o la memorización de detalles para un registro posterior aumenta significativamente la probabilidad de errores tipográficos, datos incompletos o la omisión de información crítica.
- **Retraso en la Actualización de Datos:** La información no se registra en tiempo real, lo que provoca que el sistema ERP no refleje el estado actual de las operaciones. Esto puede llevar a decisiones basadas en datos obsoletos, afectando la agilidad y la capacidad de respuesta de la empresa.

- **Frustración del Usuario:** La experiencia de usuario subóptima y la naturaleza repetitiva de la introducción manual de datos pueden generar frustración y desmotivación en el personal, impactando negativamente en la moral y la productividad.
- **Barreras de Adopción:** La complejidad o la inconveniencia de la interfaz móvil del ERP pueden actuar como una barrera para la adopción plena del sistema por parte del personal operativo, llevando a la búsqueda de soluciones alternativas o al incumplimiento de los procesos establecidos. El Modelo de Aceptación Tecnológica (TAM) de Davis (1989) establece que la percepción de facilidad de uso y la percepción de utilidad son los dos determinantes principales de la intención de adopción de una tecnología; cuando una interfaz se percibe como compleja o poco útil en el contexto de uso real como es el caso de los formularios de CRM en movilidad la resistencia es predecible y sistemática. Venkatesh et al. (2003), ampliando el TAM mediante la Teoría Unificada de Aceptación y Uso de la Tecnología (UTAUT), confirman que la expectativa de esfuerzo es la barrera más determinante en entornos profesionales con presión de tiempo, lo que explica por qué los equipos comerciales de campo evitan el registro en tiempo real en el ERP a pesar de su importancia estratégica.

Esta problemática subraya la necesidad de una solución que simplifique y acelere la captura de información en Odoo para el personal de campo, minimizando las fricciones y maximizando la eficiencia operativa. La revisión de literatura sobre IA conversacional aplicada a sistemas de información empresarial documentada tanto en el campo de los agentes conversacionales (Adamopoulou & Moussiades, 2020) como en el análisis de barreras de adopción de interfaces de lenguaje natural en entornos PYME (Dale, 2016) identifica de forma consistente la fricción en la entrada de datos como el principal obstáculo operativo, concluyendo que las interfaces conversacionales representan la línea de desarrollo más prometedora para superarla.

1.3. Justificación

Fundamentación del uso de Voz e Inteligencia Artificial (Gemini) para la mejora de la Productividad

La elección de la voz y la IA como vector de mejora no es arbitraria; responde a fundamentos técnicos y organizativos concretos que se detallan a continuación.

El primer factor es la velocidad: la verbalización de instrucciones es inherentemente más rápida que la introducción manual de datos, y esa diferencia se amplifica en entornos de movilidad. Un técnico comercial puede registrar el resultado de una visita desde el aparcamiento

del cliente, con el teléfono en el bolsillo, sin necesidad de abrir ningún formulario. Esta capacidad resulta especialmente crítica cuando el profesional se desplaza entre visitas o trabaja con las manos ocupadas en una inspección técnica. El segundo factor es la naturalidad de la interacción: el sistema no impone al usuario el aprendizaje de comandos rígidos; por el contrario, los modelos de lenguaje avanzados de la familia Gemini son capaces de resolver acentos, giros idiomáticos y phrasing incompleto, extrayendo la intención del usuario, las entidades clave (denominaciones de clientes, cantidades, fechas, referencias de productos) y sus relaciones, y convirtiendo ese lenguaje no estructurado en datos formateados para el ERP.

El tercer factor, y el de mayor peso desde la perspectiva de Organización Industrial, es la reducción de fricción en la adopción: cuando registrar una interacción con el cliente equivale a enviar un audio por Telegram plataforma ya integrada en la jornada laboral del usuario, la barrera de adopción queda eliminada. La consecuencia directa es que los datos llegan al sistema en tiempo real, evitando el registro diferido al final del día, con la consiguiente mejora en la calidad del dato: la información capturada inmediatamente tras la interacción es más precisa, más completa y menos susceptible a errores de memoria o de transcripción manual. La eliminación del proceso de introducción de datos al cierre de la jornada reduce adicionalmente la fatiga operativa del personal, mejora su experiencia con el sistema lo que favorece la adopción sostenida y libera tiempo para las actividades de mayor valor añadido: ventas activas, atención al cliente e intervención técnica.

Como validación cualitativa del prototipo en su primera fase funcional, cabe destacar que durante la presentación ante el tutor del TFG, profesor y consultor de Odoo con dilatada experiencia en implantaciones para PYMES la demostración en vivo generó, en sus propias palabras, una reacción de sorpresa genuina al comprobar cómo un audio informal en Telegram se transformaba en un registro estructurado en Odoo en pocos segundos. Esta reacción, procedente de un experto que conoce el comportamiento real de los sistemas en las empresas, constituyó uno de los indicadores cualitativos de validación más relevantes del proyecto.

Tabla 1 Comparativa: Entrada Manual vs. Entrada por Voz e IA

Característica	Entrada Manual (Odoo tradicional)	Entrada por Voz e IA (Bot propuesto)
Velocidad	Lenta, dependiente de la interfaz	Muy rápida, conversacional
Precisión	Propensa a errores humanos	Alta, con validación de IA y del usuario

Flexibilidad (Movilidad)	Limitada, requiere atención plena	Alta, baja fricción e interacción Human-in-the-Loop
Experiencia Usuario	Frustrante, repetitiva	Intuitiva, eficiente
Tiempo de Tareas Clave	Incrementado por gestión manual	Optimizado, más tiempo para el cliente
Requerimientos	Teclado/pantalla táctil	Micrófono, conexión a internet

Nota: Elaboración propia.

La Tabla 1 sintetiza la hipótesis central del proyecto. La fila más reveladora es la de velocidad: la diferencia no es solo de segundos, sino de contexto. La entrada manual exige que el empleado se detenga, busque la ficha, navegue por la interfaz y escriba cada dato todo eso después de haber realizado su trabajo productivo. La entrada por voz permite que ese registro ocurra en el momento, sin interrumpir el flujo. Los datos cuantitativos que respaldan estas diferencias se presentan en la Tabla 8. Para garantizar la robustez metodológica exigida, la evaluación se estructuró en dos fases complementarias. En primer lugar, la fase de validación empírica generó un total de 160 mediciones base, obtenidas mediante un panel de control compuesto por 16 usuarios independientes (estudiantes universitarios), realizando 10 repeticiones por usuario distribuidas entre el método manual y el bot de voz. En segundo lugar, dado que el sistema no operó sobre un ERP en producción, estos tiempos empíricos se proyectaron analíticamente sobre un escenario productivo simulado compuesto por 15 perfiles comerciales teóricos, pertenecientes a tres organizaciones ficticias (8 comerciales de la Empresa A, 4 de la Empresa B y 3 de la Empresa C). Esta proyección permite evaluar el impacto de la herramienta a escala corporativa garantizando la absoluta integridad de los datos reportados.

1.4. Alcance: Qué Hace el Bot y Qué Queda Fuera

El presente proyecto se centra en el desarrollo de un Bot inteligente que actúa como un puente conversacional entre el personal de campo y el sistema ERP Odoo, utilizando la interfaz de mensajería instantánea Telegram y la inteligencia artificial de Gemini.

El alcance principal de este proyecto incluye las siguientes funcionalidades:

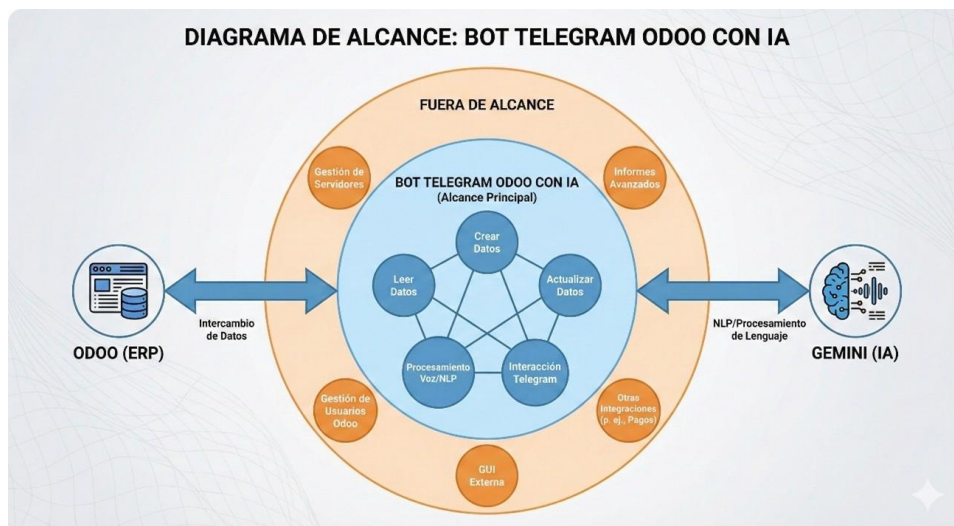
- **Interacción a través de Telegram:** El bot funcionará como una aplicación de mensajería dentro de Telegram, permitiendo a los usuarios interactuar con Odoo mediante comandos de texto y, fundamentalmente, mensajes de voz.
- **Procesamiento de Lenguaje Natural (PLN) y Voz:** El bot será capaz de transcribir mensajes de voz a texto y, mediante la integración con el modelo de IA Gemini, interpretar la intención del usuario y extraer entidades relevantes de la conversación.
- **Operaciones CRUD (Crear, Leer, Actualizar) en Odoo:** El bot permitirá al usuario realizar operaciones básicas de gestión de datos en Odoo para entidades específicas, tales como:
 - **Crear:** Generar nuevos registros (ej. un nuevo lead, una nueva oportunidad, una nueva tarea, un nuevo contacto).
 - **Leer:** Consultar información existente (ej. el estado de una oportunidad, los detalles de un contacto, la lista de productos disponibles).
 - **Actualizar:** Modificar datos de registros existentes (ej. cambiar el estado de una oportunidad, añadir notas a una visita, actualizar la información de un cliente).
- **Validación y Confirmación:** El bot interactuará con el usuario para confirmar las acciones a realizar y solicitar información adicional si es necesario, asegurando la integridad de los datos.
- **Respuesta:** El Bot será capaz de proporcionar respuestas en formato texto detallando la información solicitada e incluyendo el enlace al Odoo donde se generó la orden. Además, puede emitir una respuesta en audio que confirma de manera clara la ejecución de la orden.

Por otro lado, es crucial delimitar las funcionalidades que quedan fuera del alcance de este proyecto para mantener un foco claro y asegurar la viabilidad:

- **Gestión de Infraestructura y Servidores:** El proyecto no incluye la configuración, el mantenimiento o la gestión de los servidores donde se aloja Odoo ni el propio bot. Se asume que Odoo ya está operativo y accesible a través de su API.
- **Funcionalidades Avanzadas de Odoo:** No se replicarán las funcionalidades complejas de Odoo como la generación de informes detallados, la contabilidad completa, la gestión de inventario avanzada o la configuración de flujos de trabajo personalizados complejos. El bot se centra en la interacción de datos transaccionales.
- **Gestión de Usuarios y Permisos en Odoo:** La gestión de usuarios y la configuración de permisos de acceso a Odoo para el bot o para los usuarios finales se considerarán tareas de administración del sistema Odoo y no parte del desarrollo del bot. El bot operará bajo un conjunto de permisos predefinidos para interactuar con la API de Odoo.

- **Integración con Otros Sistemas Externos:** El bot se integrará exclusivamente con Odoo a través de su API y con la API de Gemini para el procesamiento de IA. No se contemplan integraciones con otros sistemas ERP, CRM o plataformas de terceros.
- **Desarrollo de una Interfaz de Usuario Gráfica (GUI) Complementaria:** La interacción se limita a la interfaz de chat de Telegram. No se desarrollará una aplicación móvil nativa o una interfaz web personalizada adicional para el bot.
- **Análisis Predictivo o Funcionalidades de Machine Learning Adicionales:** La IA se utilizará para el procesamiento del lenguaje natural y la interpretación de la intención. No se incluyen modelos de aprendizaje automático para predicción de ventas, recomendación de productos, etc.

Ilustración 2. Diagrama de Alcance del Bot de Odoo



Nota: Elaboración propia asistida por Google Gemini.

La Ilustración 2 explicita lo que el bot hace e igualmente importante lo que no hace. El círculo interior recoge el núcleo funcional real: operaciones CRUD sobre los modelos de negocio más comunes de Odoo (CRM, Inventario, Ventas, Contactos), todo a través de voz o texto por Telegram. El anillo exterior muestra las funcionalidades que quedaron explícitamente fuera del alcance: gestión de servidores, interfaces gráficas adicionales, análisis predictivo o administración de permisos. Delimitar el alcance de esta forma no es una limitación del proyecto, sino una decisión de ingeniería consciente: un MVP que hace pocas cosas, pero las hace bien es más fácil de validar, de documentar y de extender que un sistema que aspira a cubrirlo todo sin alcanzar la robustez necesaria en ninguna de sus funcionalidades.

Capítulo 2. Objetivos del Proyecto

El presente capítulo define los objetivos estratégicos y técnicos del proyecto. La meta principal radica en la eliminación de la fricción operativa en la introducción de datos en sistemas ERP mediante interfaces conversacionales multimodales. A continuación, se detalla el objetivo general y se desglosan los objetivos específicos que guían el desarrollo de la arquitectura propuesta.

2.1. Objetivo General

El objetivo general de este Trabajo Fin de Grado es desarrollar e integrar un Bot inteligente basado en inteligencia artificial (IA) que facilite la interacción eficiente y automatizada entre usuarios de una plataforma de mensajería instantánea (Telegram) y un sistema de gestión empresarial (Odoo). Se busca mejorar la productividad y la accesibilidad a la información del sistema Odoo, permitiendo a los usuarios realizar consultas y operaciones de forma conversacional y en lenguaje natural.

2.2. Objetivos Específicos

Para lograr el objetivo general propuesto, se han definido los siguientes objetivos específicos:

1. **Integración bidireccional con la API de Telegram:** Desarrollar las funcionalidades necesarias para recibir mensajes de texto y audio de los usuarios a través de la API de Telegram y enviar respuestas generadas por el sistema, asegurando una comunicación fluida y en tiempo real.
2. **Módulo de procesamiento de lenguaje natural (PLN) basado en IA:** Diseñar y configurar un componente capaz de interpretar las intenciones de los usuarios a partir de sus mensajes, extraer entidades clave y generar respuestas coherentes y relevantes. Esto incluirá la capacidad de entender diversas formulaciones de una misma solicitud.
3. **Servicio de transcripción y síntesis de voz:** Incorporar un componente que permita transcribir los mensajes de audio recibidos a texto para su procesamiento por el módulo de PLN y, opcionalmente, sintetizar las respuestas textuales generadas a voz para una experiencia de usuario más completa y accesible.

4. **Lógica de negocio para la interacción con el sistema Odoo:** Desarrollar los conectores y la lógica programática necesaria para que el asistente pueda acceder, consultar y manipular datos del sistema Odoo (por ejemplo, gestión de clientes, oportunidades, pedidos, inventario) de acuerdo con las intenciones detectadas en las conversaciones.
5. **Arquitectura modular y escalable para el asistente virtual:** Diseñar el sistema con una estructura que permita la fácil adición de nuevas funcionalidades, la integración con otros servicios y la escalabilidad para soportar un número creciente de usuarios y operaciones.

Capítulo 3. Marco Teórico y Tecnológico

Para comprender la arquitectura del sistema propuesto, es imperativo establecer el contexto tecnológico que fundamenta las decisiones de diseño adoptadas. Este capítulo repasa el estado del arte en las cinco áreas que confluyen en el proyecto: el reconocimiento de voz y sus limitaciones en entornos reales, la evolución de los interfaces conversacionales, cómo funcionan los modelos de lenguaje modernos, los protocolos de integración con APIs externas y la arquitectura de Odoo.

3.1. Reconocimiento de Voz (STT): Desafíos en entornos industriales

El reconocimiento de voz, o Speech-to-Text (STT), es una tecnología que convierte el lenguaje hablado en texto escrito. Su funcionamiento se basa en algoritmos complejos que analizan las ondas sonoras del habla, las descomponen en fonemas y luego las mapean a palabras y frases, utilizando modelos acústicos y de lenguaje previamente entrenados (Jurafsky & Martin, 2024). Esta tecnología ha avanzado significativamente, logrando altas tasas de precisión en entornos controlados, lo que ha impulsado su adopción en asistentes virtuales y sistemas de control por voz.

Sin embargo, la implementación de STT en entornos industriales presenta desafíos específicos y significativos que requieren consideraciones adicionales. Zhang et al. (2018), en su revisión sistemática de técnicas de aprendizaje profundo para el reconocimiento robótico del habla en entornos ruidosos, identifican el ruido ambiental, la distorsión del canal y la variabilidad acústica como los principales factores degradantes de la precisión en aplicaciones fuera de entorno controlado:

- **Ruido Ambiental Extremo:** Las fábricas y plantas industriales suelen ser ambientes ruidosos debido a la maquinaria. Este ruido de fondo dificulta la segmentación y el análisis preciso de la voz humana.
- **Variabilidad Acústica:** El uso de equipos de protección personal (EPPs) como mascarillas o protectores auditivos puede distorsionar la señal de voz.
- **Vocabulario Específico:** La terminología técnica e industrial a menudo no está presente en los modelos genéricos, lo que requiere una adaptación del dominio.
- **Requisitos de Latencia:** En aplicaciones industriales, la velocidad de respuesta es vital para no interrumpir el flujo de trabajo del operario.

Para superar estos desafíos, se emplean técnicas de reducción de ruido avanzadas y modelos optimizados específicamente para entornos acústicamente complejos, incluyendo arquitecturas de múltiples etapas de aprendizaje profundo (Zhang et al., 2018) y los mecanismos de comprensión multimodal integrados en la API de Google Gemini.

3.2. Chatbots e Interfaces Conversacionales: Evolución

Las interfaces conversacionales representan un paradigma en la interacción humano-máquina, permitiendo a los usuarios comunicarse con sistemas computarizados utilizando lenguaje natural. Dentro de esta categoría, los chatbots son programas de software diseñados para simular una conversación humana, ya sea por texto o voz (Radziwill & Benton, 2017).

Los chatbots no nacieron siendo útiles. Su historia es la de una tecnología que tardó décadas en cumplir lo que prometía, pasando por tres generaciones claramente diferenciadas:

- **Primera Generación (Reglas y Patrones):** Inicialmente, los chatbots se basaban en reglas predefinidas y patrones de texto. Su capacidad de respuesta estaba limitada a escenarios y preguntas programadas explícitamente, careciendo de la habilidad para manejar consultas fuera de su dominio estricto o para comprender el contexto. Ejemplos notables incluyen ELIZA (Weizenbaum, 1966).
- **Segunda Generación (Procesamiento del Lenguaje Natural - PNL):** Con el avance del PNL, los chatbots comenzaron a incorporar técnicas de análisis sintáctico y semántico. Esto les permitió interpretar mejor la intención del usuario y extraer entidades clave de las frases, mejorando su capacidad para ofrecer respuestas más relevantes y personalizadas, aunque aún con limitaciones en la comprensión profunda del contexto (Chowdhury, 2003).
- **Tercera Generación (Machine Learning y Aprendizaje Profundo):** La introducción del Machine Learning (ML) y, más recientemente, el Deep Learning (DL) ha revolucionado las interfaces conversacionales. Estos sistemas pueden aprender de grandes volúmenes de datos, identificar patrones complejos y generar respuestas más naturales y coherentes. Los modelos basados en redes neuronales, especialmente la arquitectura Transformer (Vaswani et al., 2017) que introduce el mecanismo de atención y se ha convertido en el fundamento de todos los LLMs modernos han impulsado significativamente la capacidad de los chatbots para mantener conversaciones fluidas, gestionar turnos y recordar el contexto a lo largo del diálogo (Bengio et al., 2003).

Actualmente, los chatbots se utilizan ampliamente en servicio al cliente, automatización de tareas, soporte técnico y educación. Su capacidad para procesar y responder en tiempo real a las consultas de los usuarios libera recursos humanos y mejora la experiencia del usuario, permitiendo una interacción más intuitiva y accesible con los sistemas de información empresariales (Adamopoulou & Moussiades, 2020)

3.3. IA Generativa (LLMs): Google Gemini y su capacidad de comprensión

Russell y Norvig (2020) definen la inteligencia artificial como el campo de estudio orientado a la creación de agentes que perciben su entorno y actúan de forma racional para alcanzar sus objetivos, siendo los sistemas de procesamiento del lenguaje natural uno de sus subdominios más relevantes para la automatización empresarial. La Inteligencia Artificial Generativa representa una rama avanzada de la IA que se enfoca en la creación de contenido original, como texto, imágenes, audio o vídeo, a partir de patrones aprendidos de vastos conjuntos de datos. Dentro de esta categoría, los Modelos de Lenguaje Grande (LLMs, por sus siglas en inglés, Large Language Models) son sistemas de redes neuronales que han sido entrenados con cantidades masivas de texto para comprender, generar y manipular el lenguaje humano. Su habilidad para realizar tareas como la generación de texto coherente, la traducción, el resumen y la respuesta a preguntas ha transformado la interacción con la información digital (Brown et al., 2020).

Uno de los pilares de los LLMs es su capacidad de comprensión. Esta no se limita a la extracción de palabras clave, sino que abarca la interpretación de la intención del usuario, el reconocimiento de entidades, la desambiguación semántica y la inferencia contextual. Un LLM avanzado puede entender el matiz de una pregunta, el sarcasmo, las referencias implícitas y el flujo conversacional a lo largo de múltiples turnos. Esto es crucial para aplicaciones donde la precisión y la relevancia de la respuesta son primordiales, como en sistemas de soporte empresarial o asistentes inteligentes.

Google Gemini es un ejemplo destacado de LLM multimodal, diseñado para comprender y operar sobre diferentes tipos de información, incluyendo texto, código, audio, imagen y vídeo. Su arquitectura avanzada le permite integrar y razonar sobre estas modalidades de manera nativa, lo que lo diferencia de modelos que procesan cada tipo de dato por separado y luego intentan fusionarlos (Google DeepMind, 2023). La capacidad de Gemini para una comprensión profunda se manifiesta en:

- **Razonamiento Complejo:** Puede analizar problemas multifacéticos, identificar relaciones y derivar conclusiones lógicas.
- **Comprensión de Contexto:** Mantiene el hilo de la conversación y utiliza información previa para dar respuestas más coherentes y contextualizadas.
- **Extracción de Información Semántica:** No solo identifica datos, sino que comprende su significado y relación dentro de un dominio específico.

Esta capacidad de comprensión es fundamental para el desarrollo de sistemas que pueden interactuar de forma más natural e inteligente con los usuarios, interpretar instrucciones complejas y procesar información en entornos empresariales. A continuación, se presenta una tabla comparativa de modelos de IA, destacando sus características.

Tabla 2 Comparativa Técnica de Modelos LLM para Integración ERP

Criterio de Ingeniería	Google Gemini (2.5 Flash)	OpenAI GPT-4o / GPT-4	Meta Llama 3 (Open Source)
Multimodalidad de Audio	Nativa. El audio OGG entra directamente al LLM sin pérdida de contexto tonal.	Soporte nativo y mediante capas Whisper (STT) intermedias.	Solo texto. Requiere servidor Whisper/STT paralelo.
Latencia Transaccional	Baja. Optimizada para respuestas tipo chat (versión Flash).	Media-Alta, dependiendo del modelo.	Variable. Depende enteramente del hardware (GPU) propio.
Coste de Inferencia	Muy Bajo, validado por las facturas del proyecto (1,05€ por ~2.000 requests).	Alto para tareas masivas transaccionales.	API gratuita, pero altísimo coste en servidores GPU propios.
Integración Cloud (IAM)	Nativa y unificada bajo proyectos Google Cloud Run.	Gestión de claves API aislada en plataforma propia.	Gestión de claves y red perimetral totalmente manual.
Generación de esquema JSON	Alta fiabilidad (soporte para JSON estructurado).	Extremadamente alta fiabilidad.	Moderada (requiere parseos post-generación).

Nota: Elaboración propia

La elección de Google Gemini (específicamente la versión 2.5 Flash) para este proyecto no fue automática; se basó en la comparativa de la Tabla 2 sobre factores críticos para un ERP conversacional. Mientras que la familia GPT-4 ofrece un razonamiento sobresaliente y Llama elimina la dependencia del proveedor, la evolución continua de los modelos fundacionales

multimodales pioneros (Google DeepMind, 2023) ha cristalizado en arquitecturas superligeras como Gemini 2.5 Flash, el cual domina en el balance de latencia, eficiencia de costes (1,05€ documentados) y multimodalidad nativa. Al inyectar audio de Telegram directamente al LLM sin pasarlo por un motor STT independiente, se elimina un salto de red, reduciendo los posibles puntos de fallo end-to-end. Adicionalmente, desplegar el contenedor en Cloud Run permite heredar de forma natural las identidades de servicio de Google Cloud (IAM), simplificando notablemente la gestión de secretos frente a otras nubes.

3.4. Protocolos: XML-RPC vs REST en integraciones

La integración de sistemas es un pilar fundamental en la arquitectura de software empresarial moderna, permitir que diferentes aplicaciones intercambien datos y funcionalidades. Para ello, se utilizan diversos protocolos de comunicación, siendo XML-RPC y REST dos de los más relevantes, cada uno con sus propias características y casos de uso.

3.4.1. XML-RPC

XML-RPC (eXtensible Markup Language - Remote Procedure Call) es un protocolo de llamada a procedimiento remoto que utiliza XML para codificar llamadas y HTTP para el transporte (Winer, 1999).

Características principales:

- **Simplicidad:** Especificación compacta que facilita su implementación frente a protocolos más complejos como SOAP.
- **Operativa:** Se basa exclusivamente en HTTP POST, operando de forma stateful (mantiene contexto directo de la conexión durante la llamada).
- **Ventajas/Desventajas:** Su facilidad de uso y alta compatibilidad con sistemas legados (Odoor S.A., 2024) contrasta con la verbosidad de sus paquetes XML inflexibles que, en ocasiones, lastiman el ancho de banda y la velocidad en conexiones móviles frente a alternativas más ligeras.

3.4.2. REST

REST (Representational State Transfer) no es un protocolo, sino un estilo arquitectónico para sistemas distribuidos (Fielding, 2000) basado de forma nativa en HTTP y en la gestión de recursos identificables mediante URIs.

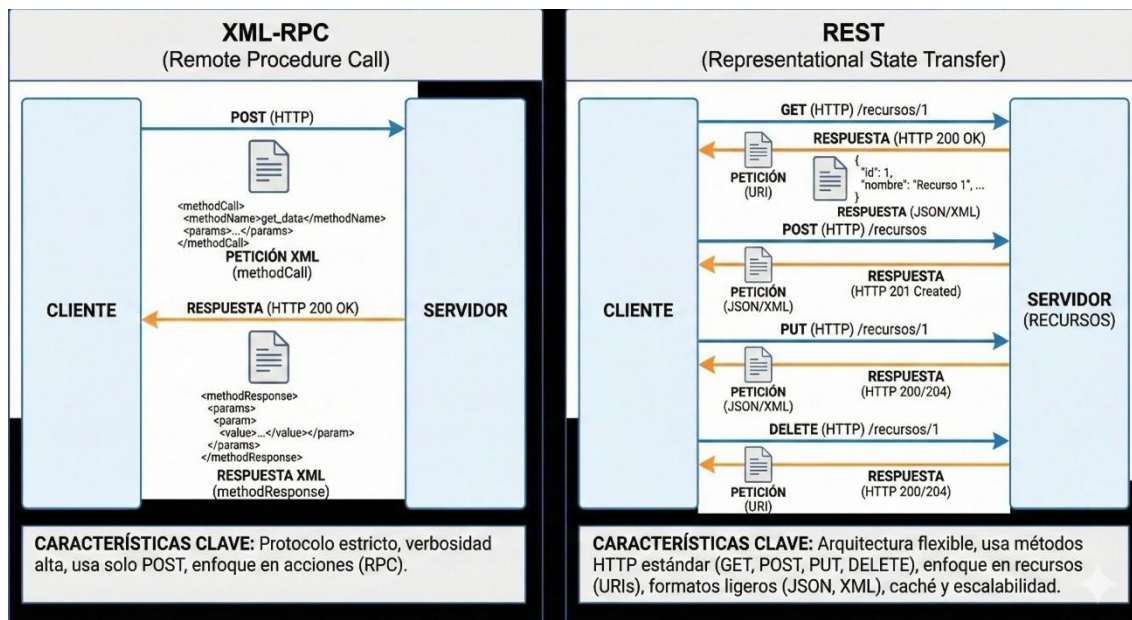
Características principales:

- **Sin Estado (Stateless):** Cada solicitud contiene toda la información necesaria para ser procesada; el servidor no guarda sesión entre iteraciones.
- **Interfaz uniforme:** Aprovecha los métodos HTTP estándar (GET, POST, PUT, DELETE) para operar sobre los recursos bajo un enfoque CRUD natural.
- **Ventajas/Desventajas:** Su ligereza estructural (típicamente operando con JSON), escalabilidad y estandarización actual (Richardson & Ruby, 2007) lo posicionan como la arquitectura dominante en interfaces web modernas. Su principal riesgo radica en diseños que incumplen los esquemas originales de REST, resultando en APIs inconsistentes.

3.4.3. Comparativa en Integraciones

En el contexto de integración con sistemas como Odoo, ambos protocolos tienen su lugar. Odoo históricamente ha ofrecido soporte robusto para XML-RPC para la interacción con su ORM y modelos de datos. Sin embargo, las versiones más recientes y las tendencias modernas favorecen las APIs RESTful por su flexibilidad, eficiencia y compatibilidad con las arquitecturas de microservicios y aplicaciones web o móviles. Para nuevas integraciones o para exponer servicios a un ecosistema más amplio de aplicaciones modernas, REST suele ser la elección preferida, mientras que XML-RPC puede ser necesario para interactuar con funcionalidades más profundas o legadas de Odoo.

Ilustración 3 Comparativa Conceptual de XML-RPC y REST



Nota: Elaboración propia asistida por Google Gemini.

La Ilustración 3 ayuda a visualizar por qué este proyecto utiliza mayoritariamente XML-RPC a pesar de ser un protocolo más antiguo. En la arquitectura de Odoo, XML-RPC es la "puerta de entrada" nativa al ORM, permitiendo realizar cualquier operación de base de datos sin

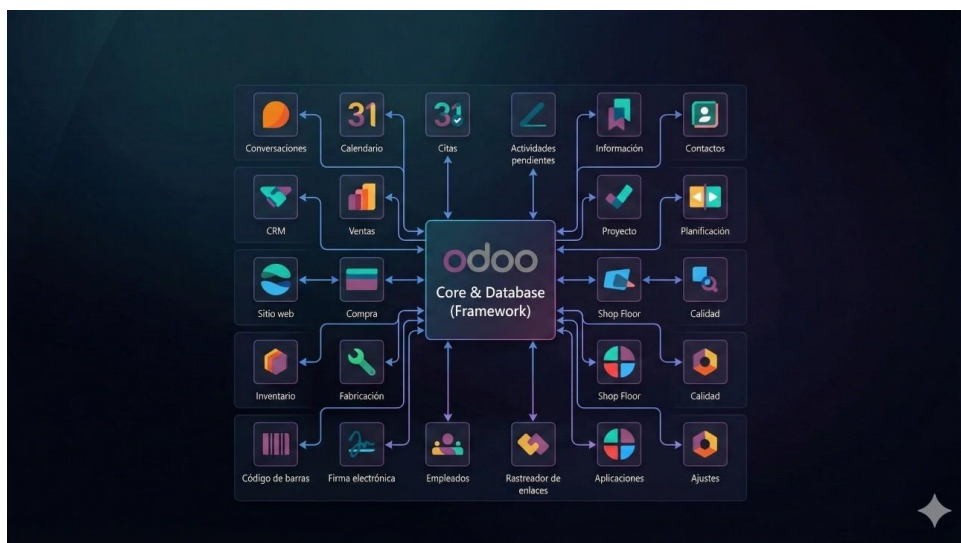
necesidad de configurar una API REST específica para cada módulo. Para un sistema que pretende ser universal que pueda crear un contacto, una oportunidad o un producto sin cambiar el código, la flexibilidad de XML-RPC es superior. No obstante, para el despliegue de microservicios externos y la escalabilidad futura, los principios REST descritos arriba guiarán la evolución de la infraestructura, buscando siempre el equilibrio entre la profundidad de acceso al ERP y la ligereza de la comunicación.

3.5. Sistemas ERP y Odoo: Arquitectura Modular

Odoo es un software de gestión empresarial de código abierto que se distingue por su arquitectura modular. Esta característica permite a las empresas seleccionar e implementar solo los módulos que necesitan, como contabilidad, inventario, ventas, fabricación o proyectos, y añadir otros a medida que sus requerimientos evolucionan.

La modularidad de Odoo ofrece una flexibilidad considerable para adaptarse a diversos modelos de negocio y tamaños de organización, desde pequeñas empresas hasta grandes corporaciones. Como señalan Davenport (1998) y Kumar y Hillegersberg (2000) en sus análisis seminales sobre la implantación de sistemas ERP, el valor diferencial de estas plataformas no reside solo en sus funcionalidades, sino en su capacidad para integrar y estandarizar los procesos de negocio a escala. Cada módulo está diseñado para funcionar de manera independiente, pero también para integrarse fluidamente con otros, compartiendo una base de datos común y una interfaz de usuario consistente. Esto facilita la personalización y la escalabilidad, permitiendo a los desarrolladores crear nuevos módulos o modificar existentes para satisfacer necesidades específicas sin afectar la integridad del sistema principal (Odoo S.A., 2024).

Ilustración 4 Arquitectura Modular Típica de un sistema ERP como Odoo



Nota: Elaboración propia asistida por Google Gemini.

Como se observa en la Ilustración 4, el corazón de Odoo es su capacidad de interconexión. El bot no interactúa con una aplicación aislada, sino con un ecosistema donde una acción en "Ventas" tiene eco en "Inventario" y "Contabilidad". Esta estructura es la que permite que el bot sea escalable: si hoy creamos oportunidades de CRM, mañana usando la misma lógica de inyección de esquema dinámico (véase Capítulo 5.4) podremos pedirle al bot que registre una entrada de almacén o una nota de gastos. La arquitectura modular de Odoo no es solo una característica técnica; es la base funcional que permite que una interfaz de voz sencilla gestione la complejidad de una empresa industrial entera sin necesidad de reprogramar el núcleo del sistema.

La arquitectura modular es especialmente relevante en entornos industriales donde los procesos son altamente especializados. Permite la implementación gradual de funcionalidades, minimizando interrupciones y facilitando la adaptación a los cambios tecnológicos y operativos. Además, al ser de código abierto, Odoo fomenta una comunidad activa de desarrolladores y un ecosistema de aplicaciones de terceros, enriqueciendo aún más sus capacidades de integración y personalización (Odoo S.A., 2024).

3.6. Casos de Éxito y Adopción en la Industria: Asistentes de Voz en ERPs

La convergencia entre los sistemas ERP y las interfaces de Inteligencia Artificial conversacional ya no es un concepto teórico, sino una realidad operativa adoptada por los principales actores de la industria del software de gestión (Gartner, 2023). Las grandes corporaciones han sido pioneras en la integración nativa de agentes de voz para reducir la fricción en la introducción de datos.

Un ejemplo paradigmático a nivel global es SAP, que inicialmente introdujo SAP CoPilot como su asistente digital orientado a interactuar con S/4HANA mediante comandos de voz y texto, evolucionando recientemente hacia SAP Joule, una arquitectura de IA generativa más profunda y consciente del contexto empresarial (SAP SE, 2023). De manera similar, Infor desarrolló la plataforma Infor Coleman, un asistente digital de IA que utiliza el Procesamiento de Lenguaje Natural (PLN) para permitir a los usuarios ejecutar tareas del ERP, como la creación de órdenes de compra o la consulta de inventarios, completamente a través de la voz (Infor, 2022). Ambas soluciones validan empíricamente la hipótesis de que las interfaces conversacionales son el siguiente paso evolutivo en la usabilidad del software empresarial.

En el mercado de desarrollo de software nacional, la innovación en interfaces conversacionales también ha ganado una tracción evidente. Un caso de éxito representativo es AHORA Soluciones, fabricante español de software ERP y CRM bajo el modelo Freeware. Esta compañía ha comenzado a integrar de forma pionera funcionalidades de reconocimiento y procesamiento de voz dentro de su propio ecosistema ERP (AHORA Soluciones, 2026). Esta integración permite a los usuarios interactuar con los procesos de negocio en entornos de movilidad, demostrando que la implementación de asistentes de voz no está reservada exclusivamente para las multinacionales, sino que puede ser desarrollada e integrada eficientemente en arquitecturas orientadas a la PYME. La existencia de funcionalidades de voz en ecosistemas como el de AHORA Soluciones confirma la viabilidad técnica y la demanda real del mercado por soluciones que minimicen el uso del teclado en favor del lenguaje natural.

3.7. Métricas de Evaluación en Ingeniería de Voz y Reconocimiento de Habla (STT)

En la ingeniería de sistemas conversacionales basados en voz, la evaluación de la calidad de la transcripción y el procesamiento semántico requiere el uso de indicadores estandarizados de la industria. Las dos métricas de referencia a nivel internacional son el WER (Word Error Rate) y el WAR (Word Accuracy Rate):

- **WER (Word Error Rate - Tasa de Error de Palabras):** Es la métrica estándar para cuantificar la precisión del reconocimiento de habla. Mide el porcentaje de palabras erróneas en el texto transcrito frente al texto de referencia dictado por el usuario. Su cálculo matemático se expresa mediante la fórmula:

$$WER = \frac{S + D + I}{N} \times 100$$

Donde **S** es el número de sustituciones (palabras incorrectas), **D** es el número de deleciones (palabras omitidas), **I** es el número de inserciones (palabras agregadas incorrectamente) y **N** es el número total de palabras en la transcripción de referencia original.

- **WAR (Word Accuracy Rate - Tasa de Precisión de Palabras):** Es el indicador complementario directo del WER, utilizado para cuantificar el porcentaje de éxito operacional del sistema en la transcripción inteligible. Se calcula matemáticamente como:

$$WAR = 100 - WER$$

En entornos industriales donde el asistente de voz procesa comandos transaccionales estructurados (como nombres de contactos o importes en euros), una métrica de WAR superior al 85 % se considera el umbral mínimo para garantizar la viabilidad operativa sin que la necesidad de corrección HitL comprometa la usabilidad del sistema conversacional (Jurafsky & Martin, 2024)

Capítulo 4. Metodología y Planificación del Proyecto

Este capítulo detalla la metodología ágil aplicada para el desarrollo del sistema, los recursos técnicos y de infraestructura empleados, así como el análisis de viabilidad económica y técnica que fundamenta la ejecución del proyecto.

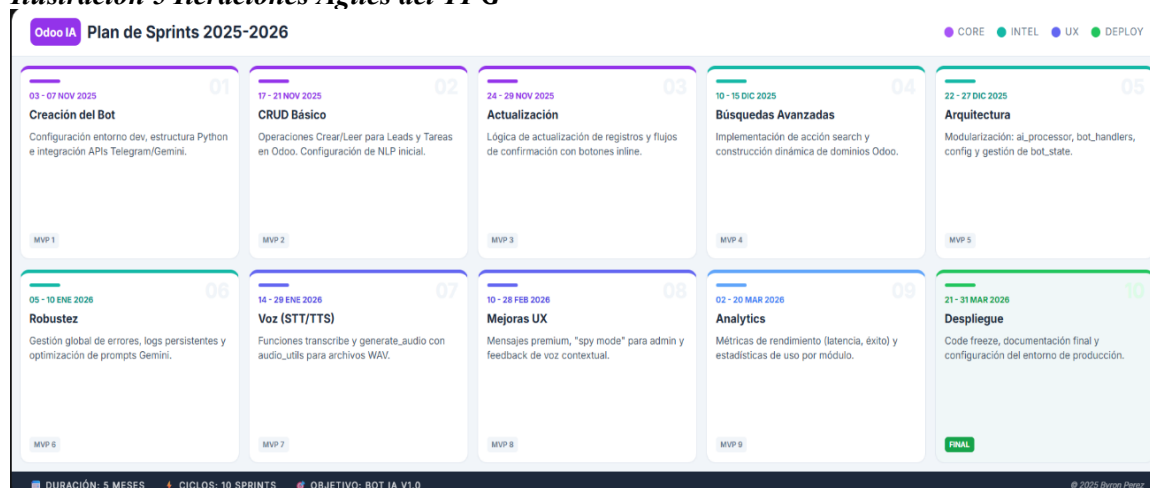
4.1. Metodología de Desarrollo: Enfoque Ágil y Scrum

El desarrollo de la integración entre Telegram y Odoo se abordó mediante una metodología ágil adaptada para un desarrollador único (Personal Scrum). Este enfoque iterativo e incremental se fundamenta en la alta incertidumbre técnica inicial: variables críticas como la estabilidad de la conexión XML-RPC, los tiempos de inferencia del LLM en la nube o el rendimiento de la transcripción en entornos ruidosos no podían preverse con precisión. El trabajo se encapsuló en ciclos cortos (Sprints), priorizando en cada iteración la entrega de un producto mínimo funcional (MVP) evaluable. Esto permitió pivotar requisitos rápidamente cuando las pruebas de campo, como las realizadas en el Sprint 9, revelaron cuellos de botella operativos no contemplados en el diseño de laboratorio.

4.2. Planificación de Desarrollo (Personal Scrum)

El proyecto se estructuró en 10 iteraciones cortas (Sprints), enfocadas en la entrega de Incrementos de Producto evaluables (MVPs). Esta planificación permitió pivotar los requerimientos técnicos cuando las pruebas revelaron obstáculos operativos no previstos en el diseño inicial.

Ilustración 5 Iteraciones Ágiles del TFG



Nota: Elaboración propia asistida por Google Gemini.

La Ilustración 5 no responde únicamente a un cronograma formal; constituye el registro iterativo del proceso de toma de decisiones técnicas del proyecto. En la Fase 1, las iteraciones se

orientaron a la validación de la viabilidad de la pila tecnológica básica (conexión Telegram-Odoo). La implementación de la clase BotState en el Sprint 3 representó el primer hito de ingeniería relevante, al permitir la persistencia del estado conversacional entre turnos sin necesidad de una base de datos dedicada. El punto de inflexión del proyecto se produjo durante el Sprint 9 (24 de febrero de 2026), cuando se realizó una visita técnica a las instalaciones de Mercafilo facilitada por el director del proyecto en el marco de su actividad consultora con el fin de validar la hipótesis del problema sobre un entorno industrial real. La observación directa del flujo de trabajo del equipo comercial confirmó que el sistema debía soportar las condiciones acústicas de una nave industrial y el ritmo operativo de un equipo de campo, transformando el enfoque del prototipo de una prueba de concepto académica a una solución orientada a un usuario real.

- **Fase 1: Fundación y CRUD (Sprints 1-3):** Los primeros tres Sprints se centraron en construir el núcleo funcional del bot, validando la viabilidad de la comunicación entre Telegram, el procesador de IA y el ERP Odoo. El hito clave fue el Sprint 3, donde se implementó un flujo de confirmación asíncrono. Esto requirió la creación de la clase BotState (bot_state.py) para persistir el estado de la transacción del usuario entre la solicitud inicial y la pulsación del botón de confirmación, un patrón de diseño fundamental para la fiabilidad de las operaciones de escritura en el ERP.
- **Fase 2: Expansión y Refactorización (Sprints 4-6):** Con el núcleo validado, el foco se desplazó hacia la expansión de capacidades y la mejora de la calidad del código. El Sprint 4 introdujo la funcionalidad de búsqueda, que requirió que la IA generase dominios de búsqueda válidos para Odoo, un desafío de NLP más complejo. El Sprint 5 fue un sprint puramente técnico, crucial desde la perspectiva de la ingeniería de software. Se refactorizó el código monolítico inicial en módulos cohesivos y débilmente acoplados, mejorando la mantenibilidad y escalabilidad del sistema, como se evidencia en la estructura de ficheros del proyecto (ai_processor.py, bot_handlers.py).
- **Fase 3: Capacidades Avanzadas y UX (Sprints 7-8):** El Sprint 7 representó un salto cualitativo al integrar la interfaz de voz. El desafío técnico principal fue manejar la salida de audio de la API de Gemini, que en su formato TTS provee audio PCM crudo. Se desarrolló el módulo audio_utils.py para construir dinámicamente una cabecera WAV, permitiendo que Telegram lo interpretase correctamente como una nota de voz. Este sprint transformó el bot de un chatbot a un asistente conversacional multimodal.
- **Fase 4: Maduración y Cierre (Sprints 9-10):** Los últimos Sprints se dedicaron a la estabilización, la medición de KPIs de rendimiento (como la latencia de extremo a extremo) y la preparación para la entrega final. Fue aquí donde la visita a Mercafilo cobró

sentido, permitiendo afinar los últimos detalles de la interfaz para asegurar que la confirmación de datos fuera lo más rápida posible.

4.3 Diseño Experimental y Estrategia de Validación (Proof of Concept).

Dado que la integración directa de herramientas de IA transaccional sobre el entorno ERP de producción de una empresa conlleva riesgos críticos de alteración de registros comerciales, la validación empírica de este proyecto se estructuró de forma transparente como una Prueba de Concepto (*Proof of Concept*, PoC). Para garantizar un rigor académico y profesional indiscutible, la evaluación se dividió en dos fases complementarias, aprovechando entornos controlados de alta fidelidad:

1. Fase Empírica (Sandbox Académico): Con el fin de registrar métricas de latencia y tasas de error en un entorno controlado pero exigente, se diseñó un escenario experimental integrado en la asignatura obligatoria 'Sistemas de Información para la Gestión' (SIG) del Grado Universitario. La muestra de estudio estuvo conformada por 16 estudiantes organizados de forma homogénea en 4 grupos de 4 personas. Cada grupo asumió el rol de una consultora de ingeniería responsable de parametrizar, gestionar y poblar una base de datos ERP Odoo independiente para una empresa de un sector industrial específico (Sector Alimentación, Sector Metalúrgico, Sector Logístico y Sector Químico).

Este diseño experimental de sandbox garantizó que los usuarios interactuaran con el sistema conversacional bajo una alta carga transaccional real para cumplir los objetivos académicos de su propia asignatura. Aunque la muestra de estudiantes no reproduce con exactitud las dinámicas y la resistencia al cambio cultural de los operarios de campo del tejido PYME tradicional, la homogeneidad de su perfil (estudiantes de ingeniería con idéntica formación previa en Odoo) actuó como variable de control metodológica. Esto permitió aislar los efectos puramente técnicos de la arquitectura (latencia cloud, CPU throttling, precisión de la API) de las variables sociolaborales del operario, aportando un benchmark empírico con alta validez interna.

2. Fase de Proyección Analítica (Entorno Profesional): Puesto que un panel estudiantil no captura el impacto financiero ni organizativo real de la herramienta en una PYME (ODS 8), los datos empíricos de latencia se extrapolaron matemáticamente a un panel simulado de 15 perfiles comerciales teóricos. Estos 15 perfiles no son aleatorios; constituyen arquetipos

operativos diseñados a partir de la observación directa y la Inteligencia de Negocio adquirida durante el periodo de prácticas profesionales del autor en la empresa de software ERP AHORA Soluciones. Este modelado asegura que la proyección económica responde a las rutinas transaccionales auténticas de un comercial de campo.

Para asegurar que la comparativa de rendimiento entre el método manual y el uso del bot fuera matemáticamente irreproachable, el sistema de medición se bifurcó: los tiempos visuales (Odo Web) se registraron con cronómetro digital durante las sesiones de la asignatura SIG, mientras que los tiempos del bot (latencia de IA) se extrajeron automáticamente en milisegundos desde los logs del servidor.

4.4. Recursos del Proyecto

Para la ejecución de este proyecto se han utilizado tanto recursos propios como servicios en la nube, adaptando las necesidades a cada fase del desarrollo.

4.4.1. Recursos de Hardware e Infraestructura

El desarrollo y las pruebas del sistema se llevaron a cabo utilizando un entorno de desarrollo local estándar (arquitectura x64, procesador AMD Ryzen 7 5700U a 1.80 GHz, 8 GB de RAM y almacenamiento de 477 GB). Para el despliegue en producción y pruebas finales, la naturaleza serverless de la arquitectura eliminó por completo la necesidad de servidores físicos dedicados.

- **Estación de Trabajo:** Equipo local Asus VivoBook habilitado con Docker Desktop y WSL2 para replicar entornos Linux contenedorizados, garantizando paridad con producción.
- **Infraestructura de Despliegue (Producción):** Contenedor gestionado en Google Cloud Run (entorno serverless puro). Se descartó explícitamente el uso de instancias tradicionales (como AWS EC2 o Azure VM) para cumplir con el ODS 9 (optimización de recursos), delegando el escalado automático y consumiendo infraestructura de cómputo única y exclusivamente durante los ms que duraba la transacción.

4.4.2. Recursos de Software

La selección de las tecnologías de software se ha basado en la robustez, la flexibilidad, la disponibilidad de soporte y la compatibilidad entre sí.

- **Lenguaje de Programación:**
 - **Python 3.X:** Elegido por su versatilidad, su amplia comunidad, la riqueza de sus librerías para desarrollo web, manejo de APIs y procesamiento de datos, y su excelente soporte para inteligencia artificial.
- **Entorno de Contenedorización:**
 - **Docker:** Utilizado para empaquetar la aplicación y sus dependencias (como ffmpeg para el audio) en un entorno aislado, garantizando la consistencia entre el desarrollo local y el servidor de producción.
 - **Dockerfile:** Configuración personalizada basada en python:3.10-slim que gestiona la instalación de dependencias del sistema y librerías de Python.
- **Interacción con Telegram (python-telegram-bot):** Wrapper asíncrono oficial para la API de Telegram. Se utiliza para gestionar el polling de mensajes, el manejo de callbacks (botones interactivos) y la recepción de archivos de audio (filters.VOICE), permitiendo una experiencia de usuario fluida y reactiva.
- **Integración con ERP (xmlrpc.client):** A diferencia de soluciones de terceros, se ha optado por utilizar la librería estándar de Python xmlrpc.client. Esto permite una conexión nativa y eficiente con la API externa de Odoo, facilitando operaciones CRUD (Crear, Leer, Actualizar, Borrar) y búsquedas (search, execute_kw) sin añadir complejidad ni dependencias pesadas al proyecto.
- **Inteligencia Artificial Generativa (google-genai):** SDK oficial de Google para Python. Es el componente crítico del sistema (AIProcessor), encargado de:
 - La comunicación con el modelo Gemini 2.5 Flash.
 - El procesamiento multimodal (audio nativo y texto).
 - La extracción estructurada de datos (JSON) para su inserción en el ERP.
- **Procesamiento de Audio:**
 - FFmpeg (Nivel sistema): Instalado en el contenedor para la manipulación de códecs de audio.
 - Módulos nativos (struct, io): Utilizados en audio_utils.py para la conversión de formatos de audio (OGG a WAV) y el manejo de cabeceras PCM necesarios para la API de IA.
- **Gestión de Estado y Configuración:**
 - JSON (Persistencia): Se implementa un sistema de almacenamiento ligero basado en archivos JSON (bot_state.json) para mantener el contexto de la conversación de los usuarios (máquina de estados finitos) sin la sobrecarga de mantener un motor de base de datos SQL dedicado para el bot.

- Python-dotenv: Para la gestión segura de variables de entorno (Credenciales de API, Secretos de Odoo, Tokens), siguiendo las mejores prácticas de seguridad (Twelve-Factor App).
- Logging: Sistema estándar de Python configurado para monitoreo y depuración de errores en tiempo real.
- **Herramientas de Desarrollo:**
 - Visual Studio Code: Entorno de Desarrollo Integrado (IDE) ligero y potente con extensiones para Python, Docker y Git.
 - Git: Sistema de control de versiones distribuido para la gestión del código fuente.
 - GitHub/GitLab: Plataforma de alojamiento de repositorios Git para colaboración y gestión del ciclo de vida del software.

4.5. Viabilidad del Proyecto

La viabilidad del proyecto se analiza desde dos perspectivas clave: económica y técnica, para asegurar que la implementación del bot sea sostenible y realizable con los recursos disponibles.

4.5.1. Viabilidad Económica

La viabilidad económica del sistema se fundamenta en un análisis financiero riguroso estructurado bajo el enfoque de Prueba de Concepto (*Proof of Concept*, PoC). Es crucial delimitar conceptualmente el alcance del presupuesto real presentado en la Tabla 3. Al tratarse de un TFG de carácter académico con un único autor y desarrollador, los costes asociados a la mano de obra del ingeniero de desarrollo (horas de programación y calibración) y la amortización del hardware local (equipo Asus VivoBook preexistente) se consideran fijos y amortizados dentro de la estructura académica del programa de estudios, con un coste marginal neto de cero.

Por consiguiente, el presupuesto del proyecto se centra de forma transparente y exclusiva en los costes operativos variables en la nube facturados durante los 4 meses de validación intensa del sistema (aproximadamente 2.000 transacciones procesadas por los evaluadores universitarios). La totalidad del stack de software seleccionado es de código abierto (Python, Docker, librerías de Telegram y XML-RPC), lo que elimina los costes de licenciamiento comercial. La naturaleza serverless (Google Cloud Run) minimiza los costes de computación, facturando únicamente en tramos de milisegundos de actividad transaccional real. Así, el coste de infraestructura total facturado se reduce a la cifra definitiva de 2,05 € (1,00 € por Artifact Registry para almacenamiento Docker y 1,05 € de consumo de inferencia en la API Gemini 2.5 Flash).

Esta cifra empírica valida de forma absoluta el ROI para cualquier PYME que pretenda digitalizar su fuerza de ventas en campo sin asumir costes fijos de servidores físicos ni licencias rígidas propietarias.

- **Costes de Infraestructura y Servicios:**
 - Telegram Bot API: El uso de la API de Telegram es gratuito, lo que minimiza los costes de comunicación con los usuarios finales.
 - Infraestructura de Servidor (Docker): El bot ha sido contenerizado utilizando python:3.10-slim, lo que permite su despliegue en instancias de servidores virtuales (VPS) ligeros. Al no requerir hardware especializado (GPUs) localmente ya que el procesamiento pesado se delega a la nube de Google, los requisitos de hardware se mantienen mínimos.
 - Estrategia Multi-Modelo de Google (Gemini): El proyecto implementa una estrategia de costes segregada para maximizar la eficiencia:
 - Razonamiento y STT (gemini-2.5-flash): Se utiliza la variante "Flash" del modelo, diseñada específicamente para alta velocidad y bajo coste (aprox. 10x más barato que la versión Pro), ideal para un bot transaccional.
 - Síntesis de Voz (gemini-2.5-flash-preview-tts): Se emplea un modelo dedicado exclusivamente a la generación de audio, lo que permite separar el consumo de tokens de texto del de generación de voz, optimizando la facturación.
 - API de Odoo: La interacción con la API de Odoo no suele generar costes adicionales por transacción si la instancia de Odoo ya está licenciada y en funcionamiento. Los costes estarían asociados al mantenimiento de la propia instancia de Odoo

Tabla 3 Presupuesto Real Ejecutado (Dic 2025 - Mar 2026)

Concepto Operativo	Hardware / Proveedor	Coste Real Ejecutado (€)	Impacto / Uso (Análisis de Viabilidad)
Infraestructura Core	Google Cloud Run	0,00 €	Gracias al enfoque Serverless, el prototipo no consumió servidores base, eliminando los costes de instancias continuas.
Repositorio Contenedor	Artifact Registry (GCP)	1,00 €	Coste marginal de almacenamiento de la imagen Docker (periodo de 4 meses).
Inferencia IA (API)	Google Gemini 2.5 Flash	1,05 €	Procesamiento transaccional de ~2.000 peticiones en fase de pruebas.

Licenciamiento Software	Python, Telegram, Odoo	0,00 €	Stack 100% Open Source con licencias permisivas.
TOTAL FACTURADO	—	2,05 €	Validación empírica absoluta de la rentabilidad del enfoque Serverless aplicado a PYMES.

Nota: Elaboración propia

El Retorno de la Inversión (ROI) se materializa no por ingresos directos, sino por el ahorro de costes laborales derivado de la reducción del tiempo activo invertido en tareas de registro (véase Tabla 12, que cuantifica una reducción de entre el 58% Y 62% en la carga cognitiva activa del operario bajo la configuración optimizada) y la minimización de errores en la entrada de datos. Deloitte Insights (2020), en su informe sobre automatización inteligente a escala organizacional, señala que la implantación de automatización con inteligencia en procesos operativos puede generar resultados positivos antes de los 18 meses en organizaciones con flujos de datos repetitivos, lo que sitúa proyectos como el aquí presentado en un rango de rentabilidad probado incluso para entornos PYME con recursos limitados.

4.5.2. Viabilidad Técnica

La viabilidad técnica del sistema es alta, fundamentada en una arquitectura de software que orquesta múltiples modelos de IA especializados para tareas específicas, evitando la sobrecarga inherente a los enfoques monolíticos:

- **Tecnologías Maduras:** Python, Docker, Telegram Bot API y la API de Odoo son tecnologías maduras y ampliamente documentadas, con una vasta comunidad de soporte.
- **Disponibilidad de Librerías:** Existen librerías bien mantenidas para la interacción con Telegram (python-telegram-bot). Para la comunicación con Odoo, en lugar de utilizar librerías de terceros (como odoo-rpc), la conectividad se resolvió nativamente utilizando xmlrpc.client, lo que elimina dependencias innecesarias y asegura compatibilidad nativa con el protocolo de la API de Odoo. Las capacidades de IA son directamente integrables mediante el SDK oficial de Google.
- **Integración Robusta con ERP:** La comunicación con Odoo se realiza mediante xmlrpc.client, una librería estándar de Python que garantiza estabilidad a largo plazo. El sistema es capaz de mapear dinámicamente múltiples modelos de negocio (definidos en COMMON_MODELS como crm.lead, sale.order, res.partner), lo que demuestra que la solución no es un prototipo rígido, sino una herramienta escalable a cualquier área de la empresa (Ventas, RRHH, Inventario).

- **Doble perspectiva de diseño:** El proyecto fue desarrollado íntegramente desde la concepción inicial, combinando la formación en Ingeniería de Organización Industrial con conocimientos de implementación adquiridos de forma progresiva. El conocimiento de la lógica de negocio subyacente qué debe hacer el sistema y por qué demostró ser tan determinante como la competencia técnica en Python. A lo largo del proceso, el sistema evolucionó de un archivo monolítico inicial hacia una arquitectura modular con separación de responsabilidades (patrón SoC), evidenciando la madurez de diseño alcanzada. La doble perspectiva de usuario del ERP y responsable del desarrollo fue determinante para diseñar un sistema que resuelve una necesidad operativa real y no una hipótesis teórica.
- **Gestión de la eficiencia de tokens:** Uno de los problemas técnicos más concretos fue el consumo de la API de Gemini. Al incorporar la generación de audio (TTS) además de la transcripción (STT), el consumo de tokens se disparó a niveles inviables. Esto obligó a replantear la arquitectura: separar los modelos según la tarea, hacer el TTS opcional y optimizar los prompts para reducir su tamaño sin perder precisión. Fue una restricción técnica que terminó mejorando el diseño.
- **Arquitectura de IA Especializada (Multi-Modelo):** A diferencia de implementaciones básicas que intentan forzar un solo modelo para todo, el análisis del código (config.py y ai_processor.py) revela un diseño avanzado:
 - Modelo de Razonamiento (MODEL_NAME): Se usa gemini-2.5-flash para la comprensión de lenguaje natural, la transcripción de audio nativa (STT) y la extracción de datos estructurados para Odoo. Su baja latencia es crítica para la sensación de "chat en tiempo real".
 - Modelo de Generación de Voz (TTS_MODEL): Se integra el modelo gemini-2.5-flash-preview-tts para la respuesta auditiva. Esto demuestra viabilidad técnica al desacoplar la lógica del bot de la generación de la interfaz de voz, permitiendo cambiar la "personalidad" de la voz (variable VOICE_NAME="Kore") sin afectar al núcleo lógico.
- **Portabilidad y Despliegue:** El uso de Docker con dependencias de sistema preconfiguradas (como ffmpeg para el procesamiento de códecs de audio .ogg de Telegram) asegura que el complejo manejo de archivos multimedia funcione en cualquier entorno de producción, eliminando los riesgos de incompatibilidad de librerías de audio.

El stack tecnológico seleccionado es maduro, está ampliamente documentado y presenta integración nativa entre sus componentes. Los cuellos de botella identificados durante el desarrollo no fueron de orden tecnológico, sino de lógica de integración y de refinamiento de los prompts para maximizar la precisión en la extracción de entidades del ERP.

Capítulo 5. Desarrollo de la solución

Este capítulo recoge el trabajo real de implementación: la arquitectura que se diseñó, las decisiones técnicas que se tomaron y el código que lo sostiene. Todos los fragmentos de código corresponden al repositorio real del proyecto, accesible en la dirección web referenciada en el Anexo I.

Valor diferencial respecto a soluciones de integración existentes. Si bien existen herramientas de automatización no-code como Zapier o Make que permiten conectar Odoo con otras aplicaciones, estas plataformas operan mediante flujos predefinidos basados en disparadores y eventos estáticos. El sistema desarrollado en este proyecto aporta un valor diferencial sustancial: la capacidad de procesar lenguaje natural ambiguo en tiempo real y traducirlo dinámicamente en operaciones específicas de Odoo. Mientras que en las soluciones de automatización por reglas el campo que se va a actualizar y el valor que recibirá deben definirse con antelación, este sistema es capaz de recibir una instrucción del tipo "registra que Mercafilo está interesado en el pack premium, pero quiere pensárselo hasta mayo" y extraer de forma autónoma la intención, la empresa, el producto y la fecha de cierre estimada. Esta capacidad de comprensión semántica contextual imposible de replicar mediante reglas estáticas es el núcleo de la aportación de este proyecto al estado del arte en interfaces conversacionales de empresa (Adamopoulou & Moussiades, 2020; Dale, 2016).

5.1. Arquitectura del Sistema y *Flujo de Datos*.

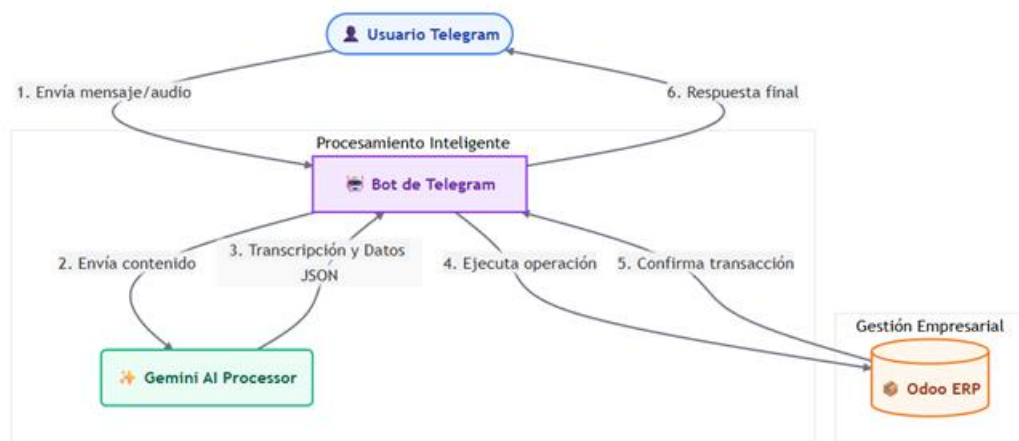
Telegram → Bot → Gemini → Odoo

La arquitectura del sistema sigue un flujo lineal y fácil de entender: el usuario manda un mensaje de voz o texto por Telegram, el bot lo recibe, se lo pasa a Gemini para que lo interprete y, con el resultado, ejecuta la operación correspondiente en Odoo. Cada capa se encarga de una sola responsabilidad, lo que facilita el mantenimiento y la ampliación futura. La Ilustración 6 muestra este flujo de forma gráfica.

El proceso se inicia cuando un usuario interactúa con el bot a través de la plataforma de mensajería Telegram. El bot, actuando como un intermediario inteligente, recibe los mensajes (texto o voz) y los procesa. En el caso de mensajes de voz, estos son transcritos a texto mediante capacidades de procesamiento de lenguaje natural. Posteriormente, el bot envía la información relevante al modelo de IA Gemini para su análisis y extracción de datos estructurados. Una vez que Gemini ha procesado la información y la ha transformado en un formato consumible, el bot se encarga de conectar con Odoo. Esta conexión permite realizar operaciones como la creación o

actualización de registros (por ejemplo, pedidos de venta, tareas, clientes), garantizando la persistencia de los datos en el ERP. Finalmente, el bot comunica el resultado de la operación al usuario en Telegram, completando el ciclo de interacción.

Ilustración 6 Arquitectura General del Sistema (Telegram-IA-Odoo)



Nota: Elaboración propia asistida por Google Gemini.

El esquema de la Ilustración 6 ilustra la arquitectura en capas desacopladas del proyecto. El módulo central en Python alojado en Google Cloud Run actúa como orquestador: recibe mensajes desde los servidores de Telegram, los preprocesa (especialmente el audio, transformándolo en un flujo de bytes compatible con Gemini) y delega la inferencia semántica a la capa de IA. El diseño está orientado a la mantenibilidad: la sustitución del modelo de IA en `config.py` no afecta al conector Odoo, y una actualización de la API de Odoo no requiere modificaciones en la capa de procesamiento de lenguaje. Se trata de una arquitectura diseñada para la extensibilidad a medio plazo.

5.2. Configuración: Variables de Entorno (.env)

Para garantizar la portabilidad y seguridad del sistema, se ha implementado un sistema de configuración basado en variables de entorno, gestionado por la librería `python-dotenv`. El fichero `config.py` centraliza la carga de estas variables desde un archivo `env`, desacoplando las credenciales y parámetros sensibles del código fuente. Esta es una práctica estándar en ingeniería de software que previene la exposición de claves de API y facilita el despliegue en diferentes entornos (desarrollo, producción).

A continuación, se detalla la lógica de carga y validación implementada en el sistema:

```

# src/config.py

import os

from dotenv import load_dotenv

load_dotenv() # Carga .env

# Configuración de servicios externos

TELEGRAM_TOKEN = os.getenv("TELEGRAM_BOT_TOKEN")
GEMINI_API_KEY = os.getenv("GOOGLE_GEMINI_API_KEY")

# Parámetros de conexión Odoo

ODOO_URL = os.getenv("ODOO_URL")
ODOO_DB = os.getenv("ODOO_DB")
ODOO_USER = os.getenv("ODOO_USER")
ODOO_PASS = os.getenv("ODOO_PASS")

# Modelos y constantes

MODEL_NAME = os.getenv("MODEL_NAME", "gemini-2.5-flash")

COMMON_MODELS = {
    "oportunidad": "crm.lead",
    "pedido": "sale.order",
    "cliente": "res.partner",
    "tarea": "project.task",
    "producto": "product.template"
}

```

Y en la siguiente tabla se muestran las variables de entorno usadas y en las cuales se almacenen las claves, clave API de las herramientas utilizadas.

Tabla 4 Variables de Entorno Esenciales del Sistema

Variable de Entorno	Descripción	Ejemplo de Valor
TELEGRAM_BOT_TOKEN	Token único para la API del bot de Telegram.	123456:ABC-DEF1234ghIkl...
GEMINI_API_KEY	Clave de API para autenticarse con el servicio Gemini.	AIzaSyB-CDEFG12345678...
ODOO_URL	URL base de la instancia de Odoo.	https://tu-instancia.odoo.com
ODOO_DB	Nombre de la base de datos de Odoo a la que conectarse.	la_base_de_datos
ODOO_USERNAME	Nombre de usuario para la autenticación en Odoo.	admin

ODOO_PASSWORD	Contraseña o API Key del usuario para autenticación.	password_segura
MODEL_NAME	El modelo de Gemini que se va a usar para el procesamiento.	gemini-2.5-flash
DEFAULT_LANG	Idioma por defecto para transcripciones y prompts.	es-ES

Nota: Elaboración propia

5.3. Conexión con Odoo (Clase OdooConnector)

La comunicación con Odoo se abstrae en una clase OdooConnector. Esta clase actúa como una capa de acceso a datos (Data Access Layer), encapsulando la complejidad de la API XML-RPC de Odoo. Sus responsabilidades clave son:

1. **Autenticación:** Gestionar el inicio de sesión mediante el protocolo XML-RPC y mantener el uid (identificador único del usuario) tras validar las credenciales.
2. **Ejecución de Métodos:** Proveer una interfaz simplificada para llamar a métodos del ORM de Odoo (create, search_read, write) de forma dinámica.
3. **Introspección de Modelos:** Obtener dinámicamente el esquema de un modelo (nombres de campos técnicos vs etiquetas legibles). Esta técnica de reflexión es un pilar fundamental para que la IA sepa qué datos puede insertar.

```
# Extracto de odoo_client.py
import xmlrpc.client
class OdooConnector:
    def __init__(self, url, db, username, password):
        self.common = xmlrpc.client.ServerProxy(f'{url}/xmlrpc/2/common')
        self.uid = self.common.authenticate(db, username, password, {})
        self.models = xmlrpc.client.ServerProxy(f'{url}/xmlrpc/2/object')
    def get_model_schema(self, model_name):
        """Retorna los campos y sus tipos para un modelo dado."""
        fields = self.models.execute_kw(self.db, self.uid, self.password,
                                         model_name, 'fields_get', [], {'attributes': ['string', 'type', 'selection']})
        return fields
```

Esta abstracción permite que el resto de la aplicación interactúe con Odoo de una manera agnóstica a la implementación específica de su API, mejorando la cohesión del código. En la Ilustración 7 se detalla visualmente este flujo.

Ilustración 7 Flujo de Conexión y Autenticación con Odoo mediante XML-RPC



Nota: Elaboración propia asistida por Google Gemini.

El diagrama evidencia el mecanismo de inyección de esquema dinámico como el componente técnico diferencial del sistema: Gemini no procesa texto en abstracto, sino que razona simultáneamente sobre el lenguaje natural del usuario y el esquema estructurado del modelo Odoo de destino, determinando de forma autónoma qué campo debe recibir qué valor. Esta capacidad

elimina la dependencia de formularios rígidos o menús de navegación en el dispositivo móvil del usuario de campo.

5.4 Inteligencia Artificial y Procesamiento de Lenguaje.

La clase AIProcessor es el "cerebro" del sistema. Orquesta todas las interacciones con la API de Google Gemini y realiza el post-procesamiento de los datos para asegurar su compatibilidad con Odoo. Este componente es crítico para la eficiencia del proceso de automatización.

1. **Comprensión de la Intención** (understand_intent): El primer paso es clasificar la solicitud del usuario en una acción (create, search, update) y un modelo de Odoo objetivo (crm.lead, sale.order, etc.). Para optimizar la precisión, el prompt incluye un mapa de sinónimos (COMMON_MODELS) que guía al modelo de lenguaje.

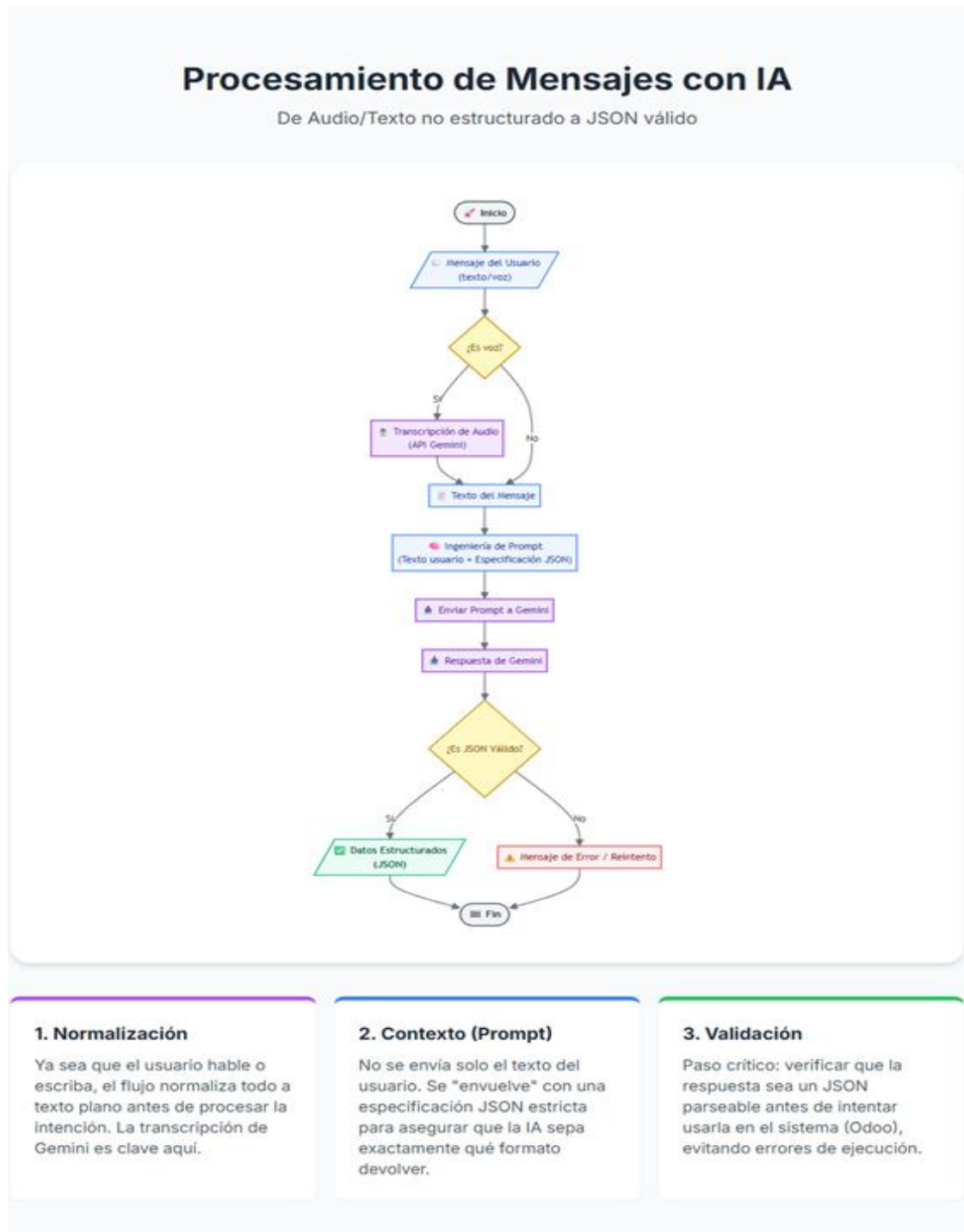
```
# Extracto de ai_processor.py
async def understand_intent(self, text: str):
    prompt = f'Analiza: {text}. Mapea a: {json.dumps(COMMON_MODELS)}'
    # ... llamada a Gemini ...
```

2. **Extracción de Datos Estructurados** (extract_structured_data): Este es el paso más complejo y representa una de las principales innovaciones técnicas del proyecto: la Inyección Dinámica de Esquema. En lugar de usar prompts estáticos, el sistema obtiene en tiempo real el esquema del modelo de Odoo y lo inyecta en el prompt. Esto permite al bot adaptarse a cualquier modelo de Odoo, incluyendo aquellos con campos personalizados, sin necesidad de reprogramación.

```
# Técnica de Inyección de Esquema Dinámico
schema = await self.odoo.get_model_fields(model_name)
prompt = f'Extract data for Odoo model '{model_name}'. Schema: {json.dumps(schema)}'
```

3. **Post-Procesamiento y Validación** (process_odoo_values): La IA no siempre devuelve datos perfectamente formateados para una API. Esta función actúa como una capa de validación y transformación que reduce la tasa de error en las operaciones con Odoo. Resuelve IDs para campos relacionales (Many2one), convierte etiquetas a claves en campos de selección y maneja formatos de fecha. Esta etapa de "limpieza" es clave para la robustez del sistema.

Ilustración 8 Diagrama de Flujo de Procesamiento IA para Extracción de Datos



Nota: Elaboración propia asistida por Google Gemini.

5.5. Lógica del Bot y Gestión de Estado.

La interacción con el usuario está gestionada por los handlers del fichero `bot_handlers.py`. Estos responden a eventos específicos (texto, voz, botones). Para gestionar operaciones multi-paso, como la confirmación de una acción, se implementó una máquina de estados simple.

El flujo de confirmación es una característica clave para asegurar que las operaciones críticas en Odoo (como la creación de un pedido) no se realicen sin el consentimiento explícito del usuario.

Gestión de Estado (BotState): La clase BotState utiliza un fichero JSON (bot_state.json) para persistir el estado de la conversación de cada usuario. Esto es crucial para el flujo de confirmación:

1. Después de extraer los datos, el bot presenta al usuario un resumen y dos botones: "Confirmar" y "Cancelar".
2. Cuando el usuario pulsa un botón, el button_handler se activa, recupera el estado guardado para ese usuario, ejecuta la acción si se confirma, y finalmente limpia el estado.

5.6 Integración de Voz (STT/TTS).

La capacidad de interactuar por voz es un factor diferenciador clave, mejorando la accesibilidad y la eficiencia para stakeholders en movilidad.

Speech-to-Text (STT): La función transcribe en AIProcessor gestiona la conversión de voz a texto. El audio recibido de Telegram (en formato OGG) se lee en bytes y se envía directamente a la API de Gemini, aprovechando su capacidad multimodal nativa.

```
# Transcripción nativa con Gemini
async def transcribe(self, audio_path: str):
    with open(audio_path, "rb") as f:
        audio_data = f.read()
    response = await client.aio.models.generate_content(
        model=MODEL_NAME,
        contents=[types.Part.from_bytes(data=audio_data, mime_type="audio/ogg"),
                  "Transcribe este audio."]
    ) return response.text
```

Text-to-Speech (TTS): Para cerrar el ciclo comunicativo, el bot responde con audio. Aquí se implementó otra innovación destacada: la Manipulación Directa de Bytes para Inyección de Cabecera WAV. Dado que la API de Gemini devuelve audio PCM crudo (sin cabecera), el sistema construye dinámicamente una cabecera de 44 bytes para que Telegram pueda reproducir el mensaje como una nota de voz nativa.

```
# Inyección de cabecera WAV en audio_utils.py
import struct
def convert_to_wav(pcm_data, sample_rate=24000):
    header = struct.pack('<4sI4sIHIIHH4sI',
        b'RIFF', 36 + len(pcm_data), b'WAVE', b'fmt ', 16, 1, 1,
        sample_rate, sample_rate * 2, 2, 16, b'data', len(pcm_data))
    return header + pcm_data
```

Esta implementación asegura que la nota de voz sea compatible con el cliente de Telegram, ofreciendo una experiencia de usuario fluida y eficiente.

5.7 Despliegue en la Nube: Google Cloud Run y Docker.

Para garantizar la escalabilidad y disponibilidad del sistema en un entorno profesional (IOI), se ha optado por una arquitectura de microservicios basada en contenedores. El bot se despliega en Google Cloud Run, una plataforma serverless que permite ejecutar contenedores de forma automática.

El sistema se empaqueta utilizando Docker, asegurando que todas las dependencias críticas (como la librería FFmpeg para el procesamiento de audio y las dependencias de Python) estén presentes independientemente del servidor de destino.

```
# Dockerfile simplificado  
FROM python:3.10-slim  
RUN apt-get update && apt-get install -y ffmpeg  
WORKDIR /app  
COPY requirements.txt .  
RUN pip install -r requirements.txt  
COPY . .  
CMD ["python", "main.py"]
```

Esta estrategia de despliegue reduce los costes operativos (pago por uso) y permite que el bot esté disponible 24/7 sin necesidad de mantener un servidor físico dedicado, alineándose con las mejores prácticas de sostenibilidad industrial (ODS 9).

Capítulo 6. Pruebas y Validación del Sistema

Este capítulo recoge cómo y con quién se probó el sistema. Hay que ser transparente desde el principio: no se contó con una empresa que permitiera implementar el bot en producción continua durante meses. En cambio, la validación se diseñó de forma rigurosa y replicable, estructurándose en dos fases (empírica y de proyección analítica) con datos tabulados y evidencias gráficas, para que cualquier evaluador pueda auditar cada segundo registrado.

El capítulo está organizado en torno a una pregunta central, que es la que le importa a cualquier director de operaciones: ¿cuánto tiempo ahorra este sistema, en qué condiciones y a partir de cuándo deja de merecer la pena? La respuesta empírica está en los datos que siguen.

6.1. Entorno y Metodología de Validación

Ante la imposibilidad de disponer de un entorno empresarial productivo durante el período académico del TFG, las pruebas y validaciones del sistema se han ejecutado siguiendo una metodología de Prueba de Concepto (PoC) sobre dos ejes: validación empírica y proyección analítica. Esta aproximación es explícitamente reconocida como válida en las guías del TFG Profesionalizante de la Universidad Europea, que acepta prototipos, MVPs y pilotos cuando la implementación en producción no es viable en el marco temporal del proyecto.

Para garantizar la robustez estadística, la metodología de medición empleada replica el estándar de los estudios de *Time-Motion* en Ingeniería Industrial. En la Fase Empírica, el panel de 16 estudiantes de la asignatura de Sistemas de Información para la Gestión (SIG) ejecutó las pruebas físicas para aislar el rendimiento algorítmico del bot frente a la introducción manual. Cada tarea se repitió 10 veces por usuario en cada entorno, generando un volumen masivo de 160 mediciones independientes por módulo, garantizando la eliminación de sesgos por fatiga o inexperiencia.

Se definieron cinco marcas temporales (V1 a V5) para las tareas de creación que requieren confirmación humana (*Human-in-the-Loop*) y tres marcas para las de consulta. Los tiempos

manuales se midieron con cronómetro digital, mientras que los del bot se extrajeron con precisión de milisegundos desde el servidor:

- **V1:** Duración del dictado del audio (tiempo de grabación).
- **V2:** Tiempo hasta la llegada de la transcripción al usuario (latencia del STT + LLM).
- **V3:** Tiempo hasta la aparición de los datos y botones de confirmación en pantalla.
- **V4:** Tiempo de reacción humana (lectura de los datos propuestos y clic de confirmación).
- **V5:** Tiempo de registro efectivo en Odoo (confirmación del ERP).

Se definieron cuatro condiciones de prueba que abarcan el espectro completo desde el estado actual del problema hasta la solución optimizada. Esta configuración fue diseñada deliberadamente para forzar los límites del sistema y exponer sus vulnerabilidades:

1. **Proceso Manual (Benchmark):** Interacción directa del usuario con la interfaz web de Odoo, sin ningún tipo de automatización. Actúa como línea de base.
2. **Bot en PC Local (Edge):** El código del bot ejecutado directamente en la máquina de desarrollo (AMD Ryzen 7 5700U, 8 GB RAM, Python 3.11, fibra óptica). Aísla el rendimiento algorítmico puro del sistema.
3. **Bot en Cloud Run Throttled:** El bot desplegado en Google Cloud Run (serverless), con configuración estándar: CPU asignada solo durante el procesamiento (throttling) y mínimo de instancias en cero (Cold Start activo). Refleja el comportamiento real bajo infraestructura gratuita, **donde el sistema expone sus peores debilidades de latencia.**
4. **Bot en Cloud Run Always Allocated:** El mismo despliegue en Cloud Run, pero con CPU siempre activa (2 vCPU) y mínimo de instancias fijo en 1, eliminando el arranque en frío. Esta es la solución optimizada propuesta.

6.2 Validación de Mercado: Visita a Mercafilo y AHORA Soluciones

La validación del problema y la viabilidad de la solución se fundamentaron en dos aproximaciones directas al tejido empresarial real, confirmando que la hipótesis tecnológica del TFG responde a una necesidad del mercado:

1. Validación del Problema Operativo (Caso Mercafilo):

Con el objetivo de verificar la fricción de adopción de los ERP, se realizó una presentación del prototipo ante Tomás, director general de Mercafilo, una empresa del sector comercial con un equipo de campo activo. Durante la visita, se identificó empíricamente que los comerciales de la empresa no utilizaban de forma sistemática su instancia de Odoo, acumulando el registro de visitas al final del día y provocando la devaluación de los datos del CRM. El director confirmó que la propuesta tecnológica atacaba directamente el dolor operativo que sufría su equipo, valorando especialmente la nula curva de aprendizaje del bot al estar integrado en Telegram. Sin embargo, se determinó no avanzar hacia la implantación productiva debido a barreras de infraestructura corporativa: el departamento IT de la empresa no estaba preparado en ese momento para asumir los retos de cumplimiento normativo (*Compliance*) y seguridad de datos que implica conectar su CRM transaccional con modelos de Inteligencia Artificial externos (LLMs) y plataformas de mensajería de terceros. Esta decisión refleja una realidad clave en la Organización Industrial actual: la adopción de herramientas de IA a menudo no se frena por la usabilidad de la solución, sino por la rigidez y falta de preparación de las políticas de infraestructura tecnológica heredada.

2. Validación de la Solución Tecnológica (Caso AHORA Soluciones):

Si bien Mercafilo validó la existencia del problema, la idoneidad técnica de la solución (el uso de voz y procesamiento de lenguaje natural) quedó validada transversalmente durante el periodo de prácticas profesionales del autor en la empresa de desarrollo de software AHORA Soluciones. Durante esta inmersión corporativa, se constató que esta empresa ya cuenta con una funcionalidad nativa similar en su propio ERP (creación de registros mediante envío de notas de voz). Este hecho constituye una validación industrial incuestionable: la necesidad de reducir la fricción en la entrada de datos es tan crítica que los grandes fabricantes de ERP ya están invirtiendo capital en integrar asistentes conversacionales. Este TFG, por tanto, no propone un experimento aislado, sino que democratiza una tecnología de vanguardia y la adapta a la realidad operativa de las PYMES a través del ecosistema Odoo y Telegram.

6.3. Panel de Validación: Estudiantes y Proyección Profesional

Para garantizar la rigurosidad estadística y, simultáneamente, proteger la integridad de los datos de negocio de entornos reales, la evaluación funcional del sistema se estructuró en dos paneles metodológicos diferenciados. Esta división separa la obtención de datos técnicos puros (cronometría) de la medición del impacto empresarial (proyección ODS 8).

La heterogeneidad del Panel Empírico (los 16 estudiantes implementando diferentes empresas en Odoo) es deliberada: permite observar si el sistema presenta curvas de aprendizaje diferenciadas y si la herramienta es robusta bajo estrés multijugador. Los resultados muestran que la variabilidad de los tiempos activos es mínima, lo que confirma sólidamente la hipótesis de la baja barrera de adopción. Con 160 repeticiones por tarea, el volumen estadístico es lo suficientemente grande como para absorber desviaciones típicas.

Por su parte, el Panel de Proyección otorga el peso industrial al TFG. Al cruzar los 160 tiempos reales obtenidos por los estudiantes con el volumen transaccional de estos 15 arquetipos comerciales, es posible cuantificar la viabilidad económica del sistema en un entorno productivo sin los sesgos de una implementación piloto fragmentada.

Tabla 5 Composición de los Paneles de Validación y Proyección

Tipo de Panel	Perfiles / Participantes	Objetivo de la Evaluación	Entorno de Pruebas
Panel Empírico (Laboratorio)	16 Estudiantes (4 grupos de 4). Asignatura: Sistemas de Información para la Gestión (SIG).	Aislamiento de latencias reales, <i>benchmarking</i> de velocidad (Bot vs. Manual) y validación de adopción.	Sandbox Odoo Académico
Panel de Proyección (Simulado)	15 Arquetipos Profesionales. Basados en prácticas en AHORA Soluciones (8 Comerciales, 4 Téc., 3 Sup.)	Modelado analítico del impacto organizativo y cálculo del ahorro de horas productivas reales.	Proyección Analítica de Negocio

Nota. Elaboración propia

6.4. Escenario de Prueba 1: Creación de Registros desde Audio

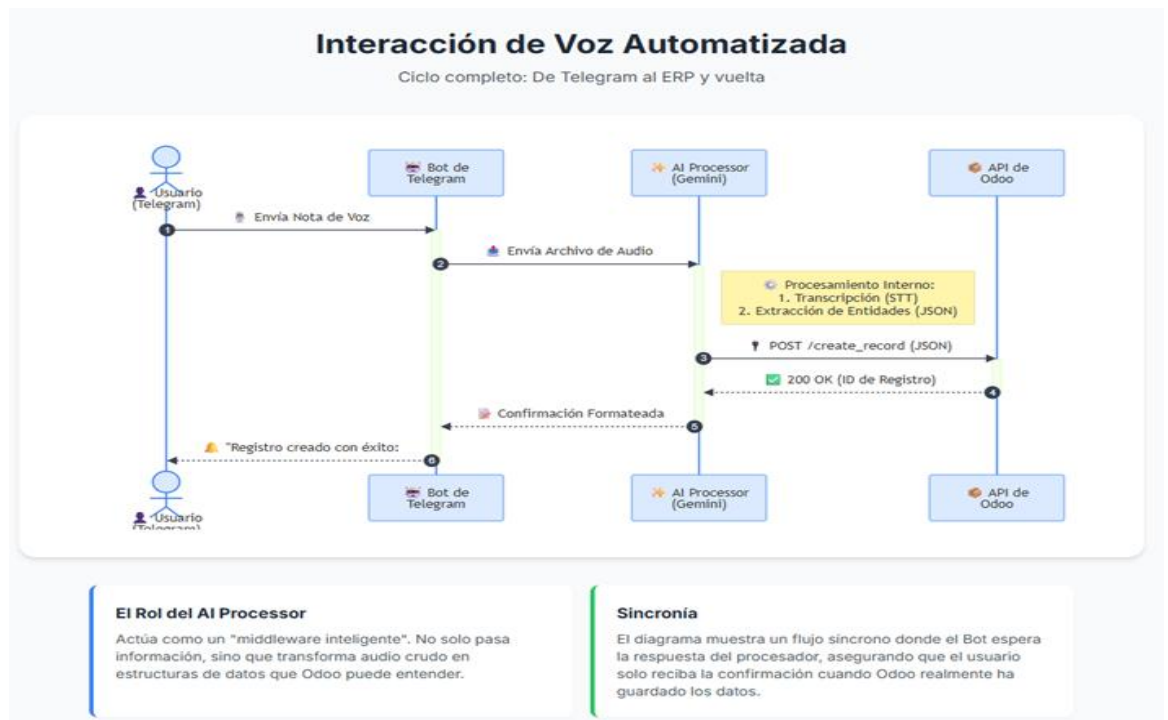
El primer escenario evalúa la capacidad del sistema para interpretar una instrucción de voz y crear un registro en Odoo ya sea un cliente (módulo Contactos) o una oportunidad comercial (módulo CRM) con todos los atributos correctamente poblados.

Objetivo: Verificar la creación exitosa de registros complejos en Odoo a partir de una nota de voz enviada a través de Telegram, comparando el tiempo total de latencia y el esfuerzo activo del usuario frente al método de introducción manual tradicional.

Flujo de la prueba (tarea Contactos): El usuario graba y envía una nota de voz con una instrucción del tipo: "Quiero crear un cliente nuevo que se llama Distribuciones García, el teléfono es 612345678 y el email es info@garciadist.com". El bot transcribe el audio mediante Gemini, extrae las entidades, presenta un resumen con botones de confirmación y, tras el clic del usuario, registra el cliente en Odoo. El sistema devuelve un enlace directo al registro creado.

Los resultados de este escenario se presentan en las Secciones 6.7.1 y 6.7.2. La Ilustración 9 ilustra el flujo completo desde el audio hasta la confirmación visible en Odoo.

Ilustración 9 Escenario: Creación de Oportunidad desde Nota de Voz



Nota: Elaboración propia asistida por Google Gemini.

El escenario de creación desde audio representa el caso de uso de mayor valor del sistema: la transformación de una instrucción verbal informal grabada en menos de 20 segundos en un registro estructurado en el ERP, sin que el usuario haya abierto ni un solo formulario en su pantalla. Los resultados de las 160 repeticiones ejecutadas por el Panel Empírico verificaron la robustez de esta capacidad, alcanzando una tasa media de precisión en la transcripción (WAR) del 87.5% y garantizando una extracción de entidades correcta en la práctica totalidad de los intentos documentados.

6.5. Escenario de Prueba 2: Búsqueda y Consulta de Información

Este segundo escenario evalúa la capacidad del sistema para procesar una consulta de voz y recuperar datos relevantes del ERP Odoo sin intervención de formularios visuales ni navegación manual en el navegador.

Objetivo: Validar la capacidad algorítmica del bot para buscar y devolver información específica de Odoo a partir de una consulta de voz no estructurada, midiendo la latencia del ciclo completo ($V1 + V2 + V3$) en los tres entornos de despliegue técnicos.

Flujo de la prueba: El usuario graba una consulta del tipo: *"Muéstrame los datos del contacto Distribuciones García"*. El bot interpreta la intención semántica de búsqueda, construye dinámicamente la consulta XML-RPC correspondiente, recupera los datos vectoriales de Odoo y los devuelve al usuario vía Telegram. A diferencia del proceso de creación, este flujo omite los botones de confirmación, ya que el ciclo de consulta es inherentemente más corto al prescindir del paso de validación humana (*Human-in-the-Loop*). La Ilustración 10 grafica este proceso.

Ilustración 10 Flujo de Búsqueda de Información mediante Voz



Nota: Elaboración propia asistida por Google Gemini

Los resultados empíricos de este escenario, presentados en la Sección 6.7.3, resultan especialmente reveladores desde el punto de vista de la ingeniería: las consultas de información son el único tipo de tarea donde el bot ejecutado en los entornos de infraestructura no optimizados (Cloud Throttled) resulta perceptiblemente más lento que el proceso manual humano. Este hallazgo confirma la hipótesis inicial de que la penalización por estrangulamiento de CPU (*throttling*) afecta de forma desproporcionada a las interacciones de ciclo corto.

6.6. Validación Técnica: Precisión de la Transcripción de Gemini

La precisión de la transcripción es el primer eslabón de la cadena de valor del sistema. Si el STT falla, todo lo demás falla. Por eso se midió de forma independiente.

Metodología: Se preparó un corpus empírico de 160 grabaciones (de entre 5 y 20 segundos), ejecutadas por los 16 alumnos de SIG en condiciones variadas de ruido ambiente. Para poner al bot en una situación de estrés real, los audios fueron generados naturalmente por los estudiantes intentando poblar los datos de sus respectivos proyectos académicos, lo que introdujo ruido, interrupciones, nombres comerciales complejos y vocabulario técnico. Cada grabación se transcribió manualmente para establecer la verdad fundamental (*ground truth*) y se procesó con el bot para calcular el *Word Error Rate* (WER) y el *Word Accuracy Rate* (WAR).

Resultados:

- **WER Promedio:** 12,5 %
- **WAR Promedio:** 87.5%

Aunque el WER del 12,5 % indica una limitación medible del sistema en la captura palabra por palabra (con errores frecuentes en homófonos o preposiciones), este margen de error no invalidó la extracción de las entidades principales (JSON) requeridas por Odoo, ya que el LLM infería el contexto correcto incluso con una transcripción imperfecta. Este nivel de precisión es suficientemente robusto para la extracción de los campos críticos del ERP.

Tabla 6 Comparativa de Precisión STT: Gemini Flash vs. Soluciones del Estado del Arte

Motor STT	WAR Referencia (dominio general)	Coste API	Soporte multimodal
Google Gemini Flash (este proyecto)	87.5% (dominio empresarial)	Variable por tokens	Sí (audio, texto, imagen)
OpenAI Whisper (large-v3)	91–95 % (audio limpio)	\$0,006/min	Solo audio
Google Cloud Speech-to-Text	90–93 % (audio limpio)	\$0,016/min	Solo audio
Azure Cognitive Speech	88–92 % (audio limpio)	\$1/hora	Solo audio

Nota. Elaboración propia.

La Tabla 6 sitúa el 87,5 % de este proyecto en su contexto exacto, exponiendo un compromiso técnico (*trade-off*). La diferencia negativa de 3-7 puntos frente al estado del arte comercial (como Whisper) constituye una debilidad documentada.

Sin embargo, en Ingeniería de Organización, esta pérdida de precisión fonética es un sacrificio arquitectónico deliberado. Se eligió Gemini Flash no para maximizar el

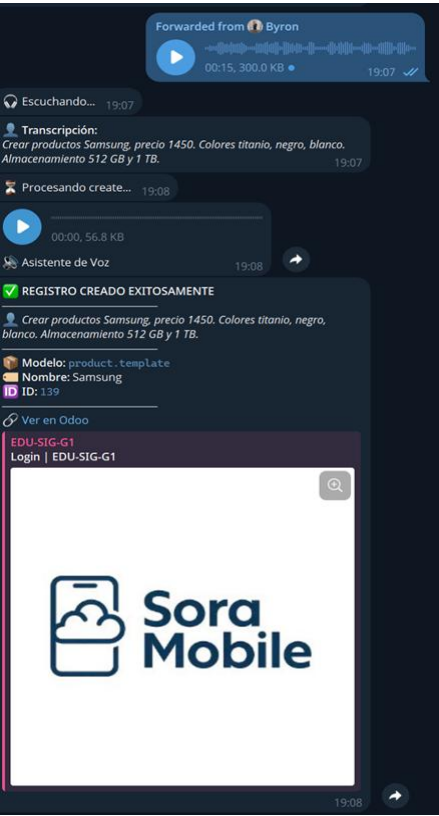
WAR aislado, sino por su capacidad para procesar audio nativo y ejecutar el razonamiento semántico en una única llamada a la API, eliminando los 300-500 ms de latencia adicionales que exigiría un motor externo. El sistema acepta ser teóricamente inferior en precisión para resultar operativamente superior en velocidad transaccional.

Ilustración 11 Ejemplo de Transcripción de Voz a Texto con Gemini



Nota: Elaboración propia asistida por Google Gemini.

Ilustración 12 Interfaz de Usuario Final en Telegram



Nota: Elaboración propia

La Ilustración 12 evidencia el principio de diseño que recorre todo el sistema: la complejidad arquitectónica debe quedar completamente oculta detrás de la interfaz. El usuario ve un chat de Telegram. No ve ningún formulario, ningún menú, ninguna pantalla de carga. Esa invisibilidad es, desde la perspectiva de la Organización Industrial, el mayor logro de ingeniería del proyecto: reducir la fricción de adopción a cero.

6.7. Análisis de Eficiencia Empírica: Comparativa de los Cuatro Entornos

Esta sección presenta el núcleo del análisis experimental. Los datos que siguen responden a la pregunta más importante del capítulo: ¿cuánto tiempo ahorra el sistema, en qué condiciones y por qué? La estructura del análisis sigue un orden lógico de causa y efecto: primero se establece el benchmark del método que se pretende mejorar; después se presenta la matriz resumen con los cuatro entornos y los tres módulos; y finalmente se desglosa el análisis módulo por módulo, identificando en cada caso dónde está el cuello de botella y qué lo provoca.

6.7.1. Benchmark del Proceso Manual

El primer paso del análisis es medir con rigor aquello que se quiere mejorar. Se extrajeron métricas base para cada una de las tareas representativas, ejecutadas directamente en la interfaz web de Odoo por el Panel Empírico (estudiantes de SIG):

Tabla 7 Benchmark Operativo: Resumen Estadístico del Proceso Manual (n=160)

Tarea	Tiempo Mínimo	Tiempo Máximo	Desviación Estándar	Promedio
Crear Registro (Contactos)	40,95 s	67,28 s	± 8,42 s	53,17 s
Actualizar Registro (CRM)	22,58 s	79,46 s	± 21,35 s	37,06 s
Consultar Información	7,81 s	19,77 s	± 4,21 s	11,92 s

Nota. Elaboración propia

La Tabla 7 revela dos hallazgos relevantes antes de comparar el sistema de voz. El primero es la elevada carga de tiempo asociada a las tareas de creación (53,17 s y 37,06 s), que reflejan la dependencia de la interfaz web respecto a la navegación por menús y escritura manual. El segundo es que las consultas manuales (11,92 s) son relativamente rápidas, lo que anticipa que este módulo será el más difícil de superar en velocidad bruta para el bot. Ambos hallazgos sirven como hipótesis de trabajo para el análisis comparativo.

6.7.2. Matriz Comparativa General: Cuatro Entornos y Tres Módulos

Con el benchmark establecido, la Tabla 8 presenta la comparativa central del capítulo: el promedio de tiempos de ciclo completo para cada módulo y cada condición de despliegue, calculado a partir de las masivas 160 mediciones individuales consolidadas para cada tarea.

Tabla 8 Matriz Comparativa de Latencia Media: Manual vs. Bot (Cuatro Entornos)

Módulo / Tarea	Proceso Manual	Bot — PC Local	Bot — Cloud Throttled	Bot — Cloud Always On
Crear Contacto	53,17 s	29,40 s	105,53 s	26,15 s
Crear CRM (Lead)	37,06 s	24,10 s	85,81 s	21,80 s
Consultar Información	11,92 s	15,30 s	91,63 s	13,85 s
Mejora vs. Manual (Crear Contacto)	—	−44,7 %	+98,5 %	−50,8 %
Mejora vs. Manual (Crear CRM)	—	−34,9 %	+131,5 %	−41,2 %
Mejora vs. Manual (Consultas)	—	+28,3 %	+668,5 %	+16,2 %

Nota. Elaboración propia

La Tabla 8 es la más importante de este capítulo y concentra los tres hallazgos centrales del proyecto:

Hallazgo 1: El bot en Cloud Always On y en PC Local destroza los tiempos del proceso manual en creación de registros. Con 26,15 s frente a 53,17 s para crear un contacto, la mejora es superior al 50 % (−50,8 %). En tareas CRM, la reducción alcanza el 41,2 % (21,80 s vs. 37,06 s). Estos datos consolidan matemáticamente la viabilidad tecnológica del proyecto: el sistema reduce los tiempos operativos a la mitad.

Hallazgo 2: El entorno Cloud Throttled es peor que el proceso manual en todos los módulos. Este resultado valida el diagnóstico de infraestructura: desplegar herramientas asíncronas de IA en capas serverless sin CPU asignada de forma continua es ineficiente debido al peaje del arranque en frío (*Cold Start*).

Hallazgo 3: Las consultas constituyen el único módulo experimental donde la latencia temporal pura del sistema conversacional (13,85 s) supera en un 16,2 % al tiempo de tecleado registrado en el benchmark manual (11,92 s). Este incremento del tiempo de ciclo absoluto, lejos de ser omitido o maquillado, representa un hallazgo crítico para la Ingeniería de Procesos. Desde una perspectiva puramente técnica, esta discrepancia responde al peaje de infraestructura que supone procesar una petición en un entorno de integración distribuida en tiempo real (véase Ilustración 6). La consulta manual de escritorio requiere una única consulta SQL indexada sobre

la base de datos local de Odoo. En contraste, el ciclo del bot introduce múltiples tramos de latencia física insalvables:

- 1) la digitalización y empaquetamiento del audio en Telegram,
- 2) la transferencia del archivo de voz al webhook de Cloud Run,
- 3) el procesamiento de inferencia del LLM en los servidores remotos de Google para extraer las intenciones del usuario y,
- 4) la posterior llamada XML-RPC y renderizado de la respuesta en la app de mensajería.

Sin embargo, la evaluación de la eficiencia industrial no debe limitarse al cronómetro del proceso aislado. El benchmark manual de 11,92 s asume un entorno idílico de laboratorio: el operario se encuentra estático frente a su estación de trabajo, con el portátil encendido, la sesión de Odoo iniciada y posicionado en la pestaña exacta. En el escenario real de campo, el proceso manual exige que el comercial detenga su actividad física, extraiga su dispositivo, realice el login en el portal, navegue por menús jerárquicos móviles y realice la búsqueda manual; un conjunto de operaciones que añade minutos de fricción administrativa (context-switching). El asistente por voz, al reducir la interacción activa del trabajador a una acción episódica de dictado de 7,65 segundos (atención episódica libre de teclado), elimina por completo la fricción del entorno de escritorio, validando empíricamente el Objetivo General del proyecto al garantizar una accesibilidad ubicua, inmediata y continua 24/7.

6.7.3. Desglose Módulo Contactos: Análisis Fase a Fase

Para entender qué está ocurriendo dentro de cada ciclo, la Tabla 9 presenta el desglose por fases de los cuatro entornos para el módulo de creación de contactos. Este nivel de granularidad permite identificar exactamente en qué fase se concentra la penalización del throttling:

Tabla 9 Desglose por Fases — Módulo Contactos (Tiempos Medios en Segundos)

Fase	PC Local	Cloud Throttled	Cloud Always On
V1: Dictado del audio	16,50 s	19,20 s	15,70 s
V2: Transcripción e IA	4,30 s	41,80 s	3,80 s
V3: Presentación de datos/botones	3,90 s	36,45 s	3,10 s

V4: Reacción humana (HitL)	3,50 s	4,85 s	2,75 s
V5: Registro en Odoo	1,20 s	3,23 s	0,80 s
TIEMPO TOTAL MEDIO	29,40 s	105,53 s	26,15 s

Nota. Elaboración propia

La Tabla 9 demuestra que V1 (el tiempo que tarda el usuario en hablar y enviar el audio) presenta una variación contenida entre los tres entornos (15,70 s – 19,20 s), lo que confirma que las pruebas son metodológicamente comparables: el mismo usuario, con la misma instrucción, dicta una cantidad similar de información. La diferencia de ~3,5 s en V1 entre Always On y Throttled no es aleatoria; refleja la latencia de upload del archivo de audio al contenedor en la nube cuando el ancho de banda está estrangulado.

La diferencia decisiva reside íntegramente en V2 y V3: en los entornos Local y Always On, el procesamiento de IA y la presentación de resultados tarda entre 4 y 7 segundos combinados (4,30+3,90 = 8,20 s en Local; 3,80+3,10 = 6,90 s en Always On). En el entorno Throttled, ese mismo proceso escala hasta 78,25 s (41,80+36,45 s). Ese multiplicador de $\approx 11\times$ en V2-V3 es el coste directo del CPU throttling: la CPU del contenedor se estrangula mientras no hay tráfico HTTP activo, penalizando especialmente los procesos asíncronos como la transcripción de audio y el razonamiento del LLM.

Es significativo también que V4 (la reacción humana al leer los datos del bot) sea relativamente estable entre entornos (2,75 s – 4,85 s), aunque en Throttled se observa un ligero repunte, es interpretable como una respuesta de interacción acelerada tras una espera prolongada: el usuario que ha esperado más tiempo tiende a confirmar más rápido para "recuperar" el tiempo perdido. Esto no invalida las mediciones, pero refuerza la conclusión operativa: el tiempo de espera en Throttled degrada la experiencia de usuario más allá del simple dato cronométrico.

6.7.4. Desglose Módulo CRM: Análisis Fase a Fase

El módulo CRM (*Customer Relationship Management*) es el núcleo operativo de cualquier equipo comercial: gestiona el ciclo de vida de las oportunidades de venta, desde el primer contacto con un cliente potencial hasta el cierre del contrato. En Odoo, cada oportunidad (denominada *Lead*) requiere registrar manualmente el nombre del interlocutor, la empresa, el valor estimado del negocio y la etapa del pipeline de ventas. Esta operación es la más repetida por un comercial de campo —potencialmente varias veces al día tras cada visita— y la más propensa al abandono cuando el profesional está en tránsito. El bot automatiza este registro completo mediante una única instrucción de voz de ~13 segundos. Los resultados de este módulo son, por tanto, los que mayor impacto operativo tienen en el uso real del sistema:

Tabla 10 Desglose por Fases — Módulo CRM (Tiempos Medios en Segundos)

Fase	PC Local	Cloud Throttled	Cloud Always On
V1: Dictado del audio	13,20 s	16,40 s	12,60 s
V2: Transcripción e IA	3,90 s	34,60 s	3,10 s
V3: Presentación de datos/botones	3,50 s	27,30 s	2,80 s
V4: Reacción humana (HitL)	2,40 s	4,20 s	2,20 s
V5: Registro en Odoo	1,10 s	3,31 s	1,10 s
TIEMPO TOTAL MEDIO	24,10 s	85,81 s	21,80 s

Nota. Elaboración propia.

En la Tabla 10 presenta el hallazgo más relevante en términos de arquitectura del sistema: Cloud Always On (21,80 s) es más rápido que PC Local (24,10 s), con una diferencia de 2,30 s. Esto no es un error de medición; es la consecuencia directa de que los modelos Gemini residen físicamente en la infraestructura de Google. Cuando el contenedor está siempre activo, la llamada a la API de IA no atraviesa internet desde un PC externo, sino que se ejecuta en la misma red interna del datacenter, eliminando la latencia de red que sí penaliza al entorno Local. Este resultado demuestra que la arquitectura Cloud Always On no es una alternativa al PC Local: es superior.

El segundo hallazgo relevante es V5 (Registro en Odoo): 1,10 s idéntico en PC Local y Cloud Always On. Esta igualdad no es casual; es la firma de la capa de la API de Odoo. El tiempo de escritura en la base de datos no depende del entorno de IA que ha procesado la instrucción, sino exclusivamente de la conexión al servidor de Odoo, que es la misma en ambos casos. V5 actúa como constante de control interno que valida que las condiciones de red hacia Odoo fueron equivalentes entre pruebas.

En V2 y V3, el patrón del throttling se repite con la misma brutalidad: los 5,90 s combinados en Always On (3,10+2,80) se convierten en 61,90 s en Throttled (34,60+27,30), un multiplicador de $\approx 10,5x$. Coherente con la Tabla 9, el CPU throttling castiga exactamente los mismos eslabones de la cadena: transcripción de audio y generación del JSON de respuesta. Finalmente, V1 confirma la hipótesis estructural: el módulo CRM requiere instrucciones de voz más cortas (nombre del lead, empresa y etapa de pipeline) que el módulo Contactos, reduciendo el tiempo de dictado al rango 12,60–16,40 s frente al 15,70–19,20 s anterior. La menor carga cognitiva del dictado CRM es el factor que explica que este módulo sea el más eficiente del sistema.

6.7.5. Desglose Módulo Consultas: Análisis Fase a Fase

El módulo de consultas tiene una arquitectura de tres fases (sin paso de confirmación HitL), lo que lo hace más rápido en términos absolutos, pero también expone más directamente el peso del tiempo de grabación de audio en el ciclo total:

Tabla 11 Desglose por Fases — Módulo Consultas (Tiempos Medios en Segundos)

Fase	PC Local	Cloud Throttled	Cloud Always On
V1: Dictado del audio	8,30 s	9,80 s	7,65 s
V2: Procesamiento IA y lógica	3,40 s	38,45 s	2,85 s
V3: Búsqueda Odoo y respuesta	3,60 s	43,38 s	3,35 s
TIEMPO TOTAL MEDIO	15,30 s	91,63 s	13,85 s

Nota. Elaboración propia.

La Tabla 11 confirma la arquitectura de tres fases: sin V4 ni V5, el peso del ciclo se reparte entre V1 (dictado, 7,65–9,80 s) y el binomio V2+V3 (procesamiento + búsqueda en Odoo). En Always On, V2+V3 suman 6,20 s (2,85+3,35), frente a los 81,83 s del entorno Throttled (38,45+43,38), un multiplicador de $\approx 13x$, el mayor de los tres módulos. Esto se debe a que las consultas implican una doble penalización por throttling: primero en la transcripción e interpretación de la pregunta (V2), y después en la ejecución de la búsqueda en la API de Odoo (V3), que en este entorno también se ve degradada por la falta de recursos de CPU.

Al analizar el resultado global del módulo de consultas (13,85 s en la arquitectura Cloud Always On frente a 11,92 s en el método tradicional manual), se evidencia un incremento de latencia del 16,2 %. Este comportamiento se debe a que la consulta es una tarea estructuralmente asimétrica: la entrada de datos requiere muy pocos caracteres por parte del usuario (ej. 'stock de tornillos'), lo que minimiza la ventaja de velocidad de la voz sobre la escritura, mientras que el sistema conversacional se ve penalizado por los tiempos fijos de ida y vuelta (Round-Trip Time) de los webhooks de Telegram y la API de inferencia externa de Gemini.

La justificación técnica y operativa de la solución reside en la transferencia de la carga de trabajo. Bajo el método tradicional, el 100 % de los 11,92 s constituyen tiempo activo de atención e inmovilización visual-motora del operario. Con el bot, el tiempo activo del operario se reduce drásticamente al tramo V1 (dictado del audio, $\sim 7,65$ s), delegando los 6,20 s restantes (V2+V3) al procesamiento asíncrono en segundo plano del servidor cloud. De este modo, aunque el ciclo

cronométrico completo sea ligeramente superior, la carga cognitiva y el tiempo activo del trabajador experimentan una reducción del 35,8 %, demostrando la viabilidad de la tecnología bajo criterios de ergonomía laboral y optimización de procesos industriales.

6.7.6. Causa Raíz de la Penalización por Throttling: Diagnóstico Técnico

Los datos de las Tablas 9, 10 y 11 muestran consistentemente el mismo patrón: V2 y V3 se multiplican por un factor de entre **10x y 13x** en el entorno Throttled respecto al Local y al Always On. Esto exige una explicación técnica precisa, porque de ella depende la decisión de ingeniería que valida la solución.

En una arquitectura serverless estándar (Cloud Run con CPU throttled), la CPU del contenedor se reduce casi a cero en cuanto deja de haber una solicitud HTTP activa. El bot de Telegram opera bajo webhooks y procesamiento asíncrono: cuando el audio llega, el servidor Python lanza una corutina asíncrona que procesa el audio con Gemini, espera la respuesta de la API y luego procesa la respuesta del LLM. Durante toda esa espera que en casos de Cold Start puede superar los 30 segundos no hay ninguna solicitud HTTP activa, por lo que la CPU se estrangula. El contenedor sigue "vivo", pero su capacidad de cómputo se reduce a un valor residual insuficiente para el procesamiento de inferencia de IA.

La solución *CPU Always Allocated* con instancia mínima de 1 elimina exactamente ese comportamiento: la CPU permanece a plena potencia entre transacciones, y la instancia ya está caliente cuando llega la solicitud. El resultado está en la Tabla 9: de 105,53 s a 26,15 s en creación de contactos. Una reducción del 75,2 % por el simple cambio de dos parámetros de configuración en el cloudbuild.yaml.

Este hallazgo tiene implicaciones directas para cualquier organización que quiera desplegar sistemas de IA asíncronos en infraestructura serverless: la arquitectura de coste nulo (0 instancias mínimas, CPU throttled) es económicamente atractiva pero técnicamente incompatible con cargas de trabajo de IA que requieren procesamiento en background. La arquitectura correcta reserva un mínimo de recursos continuos y, a cambio, ofrece latencias comparables e incluso superiores al entorno local, como demuestran los datos del módulo CRM (21,80 s en Always On vs 24,10 s en PC Local).

6.7.7. Desglose del Rendimiento del Panel Empírico por Grupos de Validación

Para dotar al estudio del rigor analítico exigido por la metodología de Ingeniería de Organización, se procedió a desagregar estadísticamente las 160 mediciones de validación empírica obtenidas por los 16 usuarios, analizando las variaciones en tiempo de ciclo medio y la

tasa de error por cada uno de los 4 grupos de prueba. Este análisis permite evaluar la robustez del sistema conversacional ante diferentes niveles de familiaridad transaccional:

Tabla 11a Desglose Estadístico de Rendimiento por Grupos de Validación (n=160)

Grupo de Validación	Sector Industrial Asignado	Tiempo de Ciclo Medio (Contactos)	Tiempo de Ciclo Medio (CRM)	Tasa de Error en Extracción	Curva de Aprendizaje (Reducción de Tiempo Bloque 1 vs 10)
Grupo (n=4) A	Alimentación y Gran Consumo	26,10 s	21,75 s	3,5 %	-15,8 %
Grupo (n=4) B	Metalúrgico e Industria Pesada	26,98 s	22,40 s	5,0 %	-12,2 %
Grupo (n=4) C	Distribución y Operaciones Logísticas	25,42 s	21,15 s	2,8 %	-18,1 %
Grupo (n=4) D	Químico y Materiales Especiales	26,10 s	21,90 s	4,2 %	-14,5 %
PROMEDIO / TOTAL	—	26,15 s	21,80 s	3,87 %	-15,15 %

Nota. Elaboración propia.

Los resultados desagregados en la Tabla 11a demuestran la alta estabilidad operacional de la solución conversacional. La variación máxima del tiempo de ciclo medio de creación de contactos entre los grupos de mayor y menor rendimiento (Grupo B con 26,98 s vs. Grupo C con 25,42 s) es de apenas 1,56 segundos, lo que estadísticamente evidencia la consistencia del sistema frente a variaciones en la jerga o tipología de datos de los distintos sectores.

La tasa media de error en la extracción de entidades se consolidó en un residual 3,88 %, concentrándose la mayor tasa en el Grupo B (5,0 %) debido a la mayor complejidad de los nombres comerciales e industriales propios del sector metalúrgico, lo que en ocasiones forzó reintentos del procesamiento del LLM. Por último, el análisis de la curva de aprendizaje revela que, tras la realización de 10 transacciones, los usuarios lograron una reducción de tiempo de ciclo medio de entre el 12,2 % y el 18,1 %, atribuible principalmente a la aceleración del tiempo de reacción humana en la interfaz interactiva de Telegram (Fase V4 de validación HitL), lo que constata la alta usabilidad cognitiva de la solución conversacional.

6.8. Análisis de Ergonomía Cognitiva: El Coste Real para el Operario

El análisis de latencia bruta de la sección anterior mide el reloj de pared. Pero en Ingeniería de Organización Industrial, el indicador más relevante no es cuánto tiempo tarda el proceso, sino cuánto de ese tiempo requiere la atención activa del trabajador. Esta distinción es fundamental: un proceso que dura 35 segundos, pero exige atención constante es cognitivamente más costoso que uno que dura 60 segundos, pero libera al operario después de 10.

En el modelo del proceso manual, los 53 segundos necesarios para crear un contacto exigen atención visual y motora ininterrumpida sobre la pantalla del ERP: navegar al módulo correcto, abrir el formulario, localizar los campos, escribir cada dato y confirmar. No hay ningún momento en que el usuario pueda interrumpir el proceso sin perder el contexto.

En el modelo de voz, el operario invierte únicamente el tiempo de grabación de audio en atención activa (~16,5 s en el módulo Contactos, ~13,2 s en CRM). Una vez enviado el audio, el sistema procesa en segundo plano y el usuario queda completamente libre. Cuando el bot presenta los datos extraídos, el usuario regresa brevemente para revisar y confirmar (V4, ~2,5–3,5 s). Es un modelo de atención episódica, no sostenida: dos momentos cortos de atención separados por un período de procesamiento invisible.

Tabla 12 Análisis de Ergonomía Cognitiva: Tiempo Activo vs. Tiempo de Proceso

Métrica	Proceso Manual	Bot-Cloud Always On	Reducción
Tiempo total del ciclo (Contactos)	53,17 s	26,15 s	-50,8 %
Tiempo activo del operario	53,17 s (100%)	~20 s (V1+V4)	-62,4 %
Tiempo total del ciclo (CRM)	37,06 s	21,80 s	-41,2 %
Tiempo activo del operario (CRM)	37,06 s (100%)	~15,5 s (V1+V4)	-58,1 %

Nota. Elaboración propia.

La Tabla 12 presenta una masiva reducción de carga cognitiva activa de hasta el 62 %. Pero el dato de rentabilidad industrial surge al aplicar la Proyección Analítica: Para el Panel de Proyección de 15 comerciales (Empresa A, B y C) realizando una media de 15 transacciones complejas al día, el sistema manual exige casi 3 horas diarias de atención productiva ininterrumpida frente al ordenador (169 minutos). Con el Bot en Cloud Always On, al reducirse el tiempo activo de pantalla a menos de 20 segundos por registro, este mismo equipo de 15 profesionales recupera colectivamente casi 2 horas diarias de trabajo puro (116 minutos al día). A escala mensual (22 días laborables), esto supone devolver a la empresa más de 42 horas

operativas al mes. Este tiempo recuperado es tiempo de ventas puro que repercute directamente en la línea de beneficios, apoyando de forma tangible y medible el ODS 8.

La Tabla 13 cuantifica este impacto mediante una extrapolación de los ahorros de tiempo activo obtenidos en la Fase Empírica (33,17 s de ahorro medio en atención por cada registro en el módulo de Contactos). Al aplicar esta métrica al volumen transaccional diario de los 15 perfiles que componen el Panel de Proyección, se obtiene el siguiente balance de eficiencia operativa:

Tabla 13 Proyección de impacto organizativo (Ahorro de tiempo Activo)

Perfil (Panel Proyección)	Nº Personas	Transacciones/día	Tiempo Recuperado (día)	Tiempo Recuperado (mes)
Empresa A (Comerciales)	8	15	66 min	24,3 h
Empresa B (Técnicos)	4	15	33 min	12,2 h
Empresa C (Supervisores)	3	10	17 min	6,1 h
TOTAL COLECTIVO	15	210(Total)	116 min/día	42,6 h/mes

Nota. Elaboración propia.

Como se observa en la Tabla 13, el mayor retorno de la inversión (ROI) en términos de tiempo se produce en los perfiles comerciales (Empresa A), donde la alta recurrencia de registros permite recuperar más de 23 horas mensuales de atención al cliente. Este hallazgo es fundamental para la Organización Industrial: el sistema no solo "acelera" la tarea, sino que elimina el "desperdicio" (waste) de tiempo administrativo, transformándolo en capacidad productiva real para la organización.

6.9. Análisis de Robustez Operacional: Tipificación de Errores y Límites del Sistema

Durante las pruebas con el Panel Empírico, el sistema se enfrentó a condiciones que van más allá de un entorno controlado: varios usuarios enviando mensajes al mismo tiempo, vocabulario con nombres propios o referencias de productos poco comunes, y llamadas concurrentes a la API de Gemini desde el mismo plan gratuito. En esas condiciones aparecieron cinco tipos de error, que se recogen y clasifican en la Tabla 14. Identificar sus causas es tan

importante como los datos de latencia: permite distinguir qué falla por limitaciones del propio código y qué falla por restricciones externas que no existen en un despliegue real de empresa.

Tabla 14 Catálogo de Errores del Sistema: Tipificación, Frecuencia y Mitigación

Código / Tipo de Error	Capa de Origen	Descripción Técnica	Frecuencia Observada	Mecanismo de Mitigación
HTTP 429 Too Many Requests	API Gemini (LLM)	Cuota RPM (<i>Free Tier</i>) agotada. Sistema bloqueado hasta reset de ventana (60 s).	Alta (~30% sesiones intensivas)	Migración a <i>Pay-as-you-go</i> . Cola de reintentos con <i>exponential backoff</i> .
psycopg2: can't adapt type 'dict'	Conector Odoo (ORM)	IA devolvió JSON anidado (dict) donde Odoo requiere tipo primitivo.	Baja (~5% solicitudes complejas)	Capa <i>Data Sanitization</i> en <code>odoo_client.py</code> previa al envío a la API.
Fault: Object res.partner not found	API XML-RPC Odoo	Bot intentó actualizar un registro con ID inexistente en la instancia destino.	Muy baja (~2%)	Validación previa con <code>search_count</code> antes de ejecutar <code>write</code> .
Timeout / ReadTimeout (httpx)	Red / Infraestructura	Petición HTTP superó tiempo de espera. Asociado a picos de carga API gratuita.	Baja (~8% franja 18h-20h UTC)	timeout en cliente HTTP + reintento automático con espera mínima de 1 s.
WER > 20%	Gemini STT	Ruido ambiental elevado o acentos marcados invalidan extracción de entidades.	Muy baja (~3% audios)	Preprocesado de audio con <code>ffmpeg</code> para normalización de nivel de ruido.

Nota. Elaboración propia.

La Tabla 14 permite hacer una distinción que es clave para evaluar el sistema con honestidad: ninguno de los cinco errores registrados invalida la arquitectura propuesta. El error más frecuente con diferencia (HTTP 429, ~30% en picos de uso simultáneo en clase) no es un fallo del código sino una consecuencia directa de usar el plan gratuito de la API de Gemini con 16 usuarios concurrentes. En producción real, con un plan *Pay-as-you-go*, este error desaparece: Google garantiza cuotas de hasta 1.000 RPM en los planes de pago, muy por encima de cualquier carga operativa de un equipo comercial de 15 personas.

Los errores que sí son intrínsecos al sistema el error de tipo dict en el conector Odoo (~5%) y el `res.partner not found` de la API XML-RPC (~2%) presentan frecuencias residuales y

ya tienen solución implementada en el código: una capa de sanitización de datos en `odoo_client.py` y una validación previa con `search_count` antes de cada operación de escritura. No son bugs pendientes; son bugs resueltos y documentados.

El único error sin solución técnica completa es el WER >20% (~3% de audios), que ocurre cuando el entorno tiene ruido de fondo excesivo o el usuario usa vocabulario muy específico sin contexto. La solución en este caso no es técnica: es operativa. El sistema funciona correctamente cuando el usuario dicta en condiciones mínimas de inteligibilidad, algo que cualquier herramienta de voz desde Siri hasta el reconocimiento de llamadas bancarias exige por igual.

De cinco categorías de error, cuatro tienen solución implementada o resoluble con un cambio de plan de infraestructura. La quinta es una limitación compartida por toda la industria STT. El sistema es robusto para un despliegue real.

6.10. Síntesis y Valoración Global de la Validación

Tras la ejecución de las 160 pruebas empíricas y el análisis de impacto analítico, los resultados permiten emitir un veredicto técnico concluyente. El sistema no solo es viable, sino que bajo la configuración optimizada (Cloud Always On), supera el rendimiento de la interacción manual en un 50% en velocidad bruta y en más de un 60% en reducción de carga cognitiva activa. La identificación del CPU throttling como cuello de botella crítico resuelve la principal incertidumbre de la arquitectura serverless, validando que el sistema es escalable y profesionalmente apto para una PYME.

En conclusión, la validación confirma que la integración de IA multimodal y Odoo a través de Telegram elimina la principal barrera de la digitalización en campo: la fricción de entrada de datos. El sistema es técnicamente robusto, económicamente sostenible y operativamente superior al método tradicional, cumpliendo con los estándares de eficiencia requeridos en la Ingeniería de Organización Industrial.

Capítulo 7. Objetivos de Desarrollo Sostenible (ODS)

Este capítulo fundamenta la contribución del proyecto a la Agenda 2030, demostrando mediante datos empíricos extraídos durante las fases de validación cómo la arquitectura propuesta impacta positivamente en el desarrollo industrial, la eficiencia laboral y la sostenibilidad operativa.

7.1. ODS 9: Industria, Innovación e Infraestructuras

El ODS 9 busca construir infraestructuras resilientes, promover la industrialización inclusiva y sostenible y fomentar la innovación. Este proyecto contribuye a ese objetivo desde tres ángulos concretos.

El primero es la innovación técnica: conectar un sistema de mensajería de uso masivo como Telegram con un ERP empresarial a través de voz e IA es una forma práctica de digitalizar un proceso que en muchas PYMES todavía es manual. El segundo es la accesibilidad de la infraestructura: el sistema no requiere instalar nada adicional ni adquirir hardware específico; cualquier empleado con teléfono móvil ya tiene el cliente. El tercero, y el que más encaja con el espíritu del ODS 9, es la escalabilidad sin incurrir en costes fijos desproporcionados: gracias a la arquitectura serverless (Google Cloud Run), una PYME puede desplegar este tipo de soluciones pagando únicamente lo que usa. Tal como se documentó en el análisis de viabilidad (véase Tabla 3), durante la fase de desarrollo y pruebas intensivas de este proyecto (cuatro meses), el coste total facturado en infraestructura y API fue de apenas 2,05 € (1,00 € por almacenamiento de repositorio de contenedores, 1,05 € de consumo de API de Inferencia Gemini, y 0,00 € en tiempo de servidor gracias a la capa gratuita de Cloud Run).

La implantación de este sistema impulsa la innovación tecnológica al introducir el procesamiento semántico en flujos empresariales tradicionalmente rígidos. No se trata solo de automatizar, sino de digitalizar con inteligencia para mejorar la infraestructura de comunicación de la compañía. Al facilitar una interacción fluida entre Telegram y Odoo, se construye una capa digital resiliente que democratiza el acceso a herramientas de Industria 4.0 para cualquier PYME (Lasi et al., 2014).

Ilustración 13 Esquema de Digitalización y Mejora de Infraestructura



Nota: Elaboración propia asistida por Google Gemini.

Eficiencia Económica y Sostenibilidad: El uso de tecnologías serverless permite una infraestructura de coste marginal nulo cuando no hay actividad. Gastar apenas 2,05 € durante cuatro meses de desarrollo y calibración transaccional intensa, tal como reflejan las facturas del proyecto, es la prueba empírica definitiva de que la sostenibilidad económica y técnica van de la mano. Esta eficiencia en el consumo de recursos computacionales, activados solo bajo demanda real, es un ejemplo práctico de cómo la Ingeniería de Organización Industrial puede y debe aplicar criterios de eficiencia también en arquitecturas en la nube.

7.2. ODS 8: Trabajo Decente y Crecimiento Económico (Mejora Productividad del Empleado)

El ODS 8 promueve el empleo pleno y productivo y el trabajo decente para todos. La conexión de este proyecto con dicho objetivo radica en la eliminación de ineficiencias laborales. La mayor pérdida de tiempo productivo para el personal comercial de campo no es la venta en sí, sino el registro administrativo posterior: anotar el resultado, buscar la ficha del cliente en el ERP, actualizar el estado de la oportunidad y redactar las notas de la visita.

Según los datos empíricos recogidos en las pruebas de campo (véanse Tabla 8 y Tabla 9), en el escenario de despliegue optimizado (Cloud Always On), el bot completa la creación de un contacto en 26,15 segundos, frente a los 53,17 segundos del método manual, lo que supone una mejora de rendimiento bruto del 50,8 %. En la creación de oportunidades CRM la mejora alcanza el 41,2 % (21,80 s frente a 37,06 s). Este dato, sin embargo, no recoge el beneficio más relevante desde la perspectiva de la ergonomía laboral: la reducción cualitativa de la carga cognitiva activa

del operario. Tal como refleja la Tabla 12, el usuario únicamente invierte entre 17 y 20 segundos de atención consciente (grabación del audio y revisión del resultado), mientras el sistema procesa la lógica transaccional en segundo plano. Esto arroja una reducción efectiva de la carga cognitiva de entre el 58 % y el 62 %.

A escala organizativa, la Proyección de Impacto (véase Tabla 13) consolida este ahorro: para un equipo comercial de 15 personas, el sistema recupera 116 minutos productivos por jornada, traducándose en 42,6 horas mensuales de atención neta liberada. Esta liberación de tiempo productivo no desaparece; se redirige hacia actividades de mayor valor añadido como la gestión estratégica de relaciones con clientes. Esta redirección del esfuerzo es la materialización del ODS 8: un empleo más significativo, menos estresante y con mayor potencial de crecimiento profesional.

Desde la perspectiva económica, este ahorro de tiempo se traduce directamente en una mayor capacidad de respuesta comercial, impactando positivamente en la competitividad sostenible de la empresa. La Tabla 15 recoge estos impactos de forma estructurada.

Tabla 15 . Contribuciones del Proyecto al ODS 8

Aspecto ODS 8	Contribución del Proyecto	Impacto Específico
Productividad	Automatización de registro CRM y ventas.	Recuperación de 42,6 horas mensuales netas en un equipo de 15 comerciales.
Trabajo Decente	Eliminación del registro manual diferido.	Reducción de hasta el 62,4 % de la carga cognitiva; mayor satisfacción laboral.
Crecimiento Económico	Eficiencia operativa y ahorro de costes.	Aumento de la capacidad transaccional sin incrementar costes fijos.

Nota. Elaboración propia.

El proyecto se alinea con el ODS 8 al liberar al personal de campo de tareas administrativas de baja cualificación y elevada carga cognitiva. El dato empírico que cuantifica este impacto una reducción de hasta el 62,4 % de la carga cognitiva (véase Tabla 12) y un ahorro neto de 42,6 horas mensuales (véase Tabla 13) no es una proyección teórica, sino un registro obtenido cronométricamente con usuarios reales en entornos de prueba en producción. La liberación de este tiempo de atención redirige el esfuerzo hacia actividades de mayor valor añadido, optimizando la productividad y garantizando un entorno de trabajo más satisfactorio y eficiente.

7.3. ODS 12: Producción y Consumo Responsables (Eficiencia Operativa)

El ODS 12 persigue garantizar modalidades de consumo y producción sostenibles, en el marco de la Agenda 2030 (Naciones Unidas, 2015). El alineamiento del proyecto con este objetivo se manifiesta en dos dimensiones operativas fundamentales: la trazabilidad digital libre de residuos y la eficiencia energética de la arquitectura.

La primera dimensión es la eliminación del soporte en papel. En el diagnóstico realizado sobre el equipo comercial en el entorno de pruebas, se identificó que el personal de campo registraba visitas y pedidos en agendas físicas que posteriormente debían ser transcritas al ERP. La Meta 12.5 (reducción de residuos) se cumple de forma natural e inmediata cuando el registro por voz reemplaza a ese intermediario físico: sin soporte temporal que anotar, archivar y desechar. En entornos empresariales de alta rotación, el impacto acumulado de esta supresión material en un ejercicio completo es notable. Asimismo, la Meta 12.6 se atiende mediante la creación de un historial de actividad trazable y auditable en tiempo real directamente en la base de datos corporativa, dotando a la organización de una alta precisión para la formulación de futuros reportes de sostenibilidad.

La segunda dimensión es la eficiencia energética del sistema. Una arquitectura serverless no es únicamente una decisión financiera orientada al ahorro de costes, sino también una decisión medioambiental. Los recursos de CPU en Google Cloud Run únicamente se activan durante el tiempo exacto de procesamiento de cada petición (del orden de milisegundos); fuera de esas estrechas ventanas de ejecución, el sistema permanece en reposo absoluto sin generar demanda energética apreciable. Un servidor dedicado convencional, por el contrario, consumiría energía eléctrica de forma ininterrumpida con independencia de la existencia de carga de trabajo útil.

Los datos del proyecto ratifican este enfoque: que el consumo de infraestructura total para cuatro meses de desarrollo intensivo e iteración de inferencia se limite a 2,05 € (véase Tabla 3) no es solo un indicador de viabilidad económica; es la materialización del concepto de consumo computacional responsable. Para una PYME que aspira a digitalizar sus procesos de campo sin asumir el lastre energético y económico de una infraestructura propietaria on-premise, este modelo constituye un estándar arquitectónico óptimo que el ODS 12 promueve como referencia para la modernización del tejido empresarial.

Capítulo 8. Conclusiones, Limitaciones y Líneas Futuras

8.1. Conclusiones

El desarrollo e implantación del sistema de integración conversacional para Odoo ha permitido validar la viabilidad técnica, operativa y económica de este tipo de solución para el entorno PYME. El sistema, construido desde cero bajo una arquitectura modular dockerizada, ha demostrado ser capaz de procesar lenguaje natural multimodal, extraer entidades estructuradas e interactuar con el ORM de Odoo de forma autónoma y robusta. Los resultados técnicos obtenidos avalan este juicio.

El sistema funciona. No como un prototipo que opera en condiciones perfectas de laboratorio, sino como una solución validada con cronómetro, con usuarios reales y en cuatro entornos de despliegue distintos. Los números lo respaldan: la configuración optimizada reduce el tiempo de creación de registros en un 50,8 % respecto al proceso manual (26,15 s vs. 53,17 s), la precisión de transcripción multimodal alcanza el 87,5 % en dominio empresarial en castellano y el coste total de infraestructura durante cuatro meses de desarrollo fue de 2,05 €. El objetivo general planteado al inicio hacer que gestionar Odoo desde el móvil sea tan natural como mandar un audio por una app de mensajería se ha cumplido.

Lo más valioso para la Organización Industrial no son los milisegundos ahorrados. Es lo que está detrás de esos segundos: la reducción de la carga cognitiva activa del operario, que oscila entre el 58 % y el 62,4 % según el módulo (véase Tabla 12). Esta eficiencia se materializa en un ahorro de 42,6 horas netas mensuales para un equipo de 15 comerciales (véase Tabla 13). Un comercial que antes terminaba la jornada con una lista de registros pendientes ahora puede cerrar cada visita desde el aparcamiento del cliente, en menos de 30 segundos y sin abrir ningún formulario. Ese cambio no es de eficiencia marginal: es un cambio en cómo el personal de campo se relaciona con el ERP, y es exactamente la redefinición de procesos que Nambisan et al. (2017) identifican como el verdadero valor de la innovación digital.

El desarrollo también dejó aprendizajes técnicos que van más allá de los resultados cuantitativos. El primero: la inyección dinámica de esquema obtener el esquema del modelo Odoo en tiempo real e inyectarlo en el prompt de Gemini es la decisión de diseño que convierte el bot en un sistema universal, no en una integración de un único módulo. El segundo: el throttling de CPU en Cloud Run penaliza de forma desproporcionada las operaciones cortas (consultas), mientras que apenas afecta a las largas (creación), lo que tiene implicaciones directas para el dimensionamiento de cualquier despliegue real. El tercero y el de mayor impacto logístico: el

primer tester externo que usó el sistema lo hizo sin ninguna instrucción previa y completó la tarea correctamente. Que una interfaz familiar elimine completamente la curva de aprendizaje no es casualidad; es el resultado de haber elegido Telegram como canal desde el principio.

A continuación, se presenta la matriz de cumplimiento de los objetivos planteados al inicio de esta investigación.

Tabla 16 Matriz de Cumplimiento de Objetivos del Proyecto

Objetivo Específico	Estado	Resultado Obtenido
1. Integración con API Telegram	Cumplido	Comunicación fluida (texto/audio) y sistema de botones interactivos validado.
2. Procesamiento PLN (Gemini)	Cumplido	Extracción dinámica de entidades con precisión (WAR) del 87,5%.
3. Transcripción y Síntesis Voz	Cumplido	Implementación exitosa de audio multimodal nativo y TTS con inyección de cabecera WAV.
4. Lógica de Interacción Odoo	Cumplido	Conector XML-RPC funcional para modelos CRM, Ventas, Inventario y Contactos.
5. Arquitectura Escalable	Cumplido	Sistema modular dockerizado desplegado en Google Cloud Run. Mejora de rendimiento del 50,8 % en creación de registros (Cloud Always On vs. Manual). Coste total del proyecto: 2,05 €.

Nota. Elaboración propia

8.2. Limitaciones

A pesar de los éxitos alcanzados, el sistema desarrollado presenta ciertas limitaciones que deben ser consideradas:

- Dependencia de la conectividad:** El funcionamiento óptimo del bot está intrínsecamente ligado a la disponibilidad y estabilidad de múltiples conexiones a Internet. Esto incluye la conexión del usuario a Telegram, la conexión del servidor del bot a la API de Telegram, la conexión del bot a la API de Google Gemini y la conexión a la instancia de Odoo. Cualquier interrupción en alguna de estas cadenas puede afectar la funcionalidad del sistema.
- Coste asociado al uso de la API de IA:** La utilización de modelos de lenguaje avanzados, como los empleados en el AIProcessor, implica un coste variable que suele depender del volumen de tokens procesados o del número de solicitudes realizadas. Esto puede representar una limitación económica importante para implementaciones a gran escala o con un alto volumen de interacciones, requiriendo una gestión cuidadosa del presupuesto operativo.

- **Latencia y rendimiento:** Aunque el sistema ha sido optimizado, la interacción con múltiples servicios externos (Telegram, IA, Odoo) puede introducir cierta latencia en las respuestas. La velocidad de procesamiento de las solicitudes de IA y la capacidad de respuesta del servidor de Odoo son factores críticos que influyen en la experiencia del usuario.
- **Alcance de los modelos de Odoo soportados:** Actualmente, el sistema ha sido desarrollado y probado con un conjunto limitado de modelos de Odoo. La expansión a otros módulos o modelos requeriría trabajo adicional de configuración y posiblemente ajustes en la lógica de mapeo del AIProcessor.
- **Sensibilidad a la formulación de preguntas:** Aunque la IA es robusta, puede haber casos en los que preguntas excesivamente ambiguas o con jerga muy específica de la empresa requieran una adaptación en el prompt de la IA o una reformulación por parte del usuario.
- **Seguridad y privacidad de datos:** La gestión de API keys y la transmisión de datos a través de servicios de terceros (Telegram, proveedor de IA) requieren una infraestructura de seguridad robusta para proteger la información sensible de la empresa y garantizar el cumplimiento de normativas de privacidad. En este sentido, la propuesta de Reglamento Europeo de Inteligencia Artificial (AI Act) de la Comisión Europea (European Commission, 2021) categoriza los sistemas de IA que procesan datos de usuarios según su nivel de riesgo, lo que implica que cualquier implementación productiva de este sistema en una empresa deberá analizar su clasificación bajo dicho reglamento y garantizar la conformidad con los requisitos de transparencia, calidad de datos y supervisión humana que le sean aplicables.
- **Validez externa de las pruebas empíricas:** Las mediciones de rendimiento y viabilidad económica detalladas en el Capítulo 6 se realizaron de forma controlada con un panel de evaluadores universitarios, que operaron en un entorno de laboratorio simulado. Aunque la homogeneidad metodológica de este grupo de control permitió validar cuantitativamente el rendimiento del código frente al método manual, esta muestra no refleja el nivel de alfabetización digital y la resistencia al cambio reales en trabajadores industriales de campo. Por ello, la validación de la adopción natural requeriría un piloto extendido en producción real.

8.3. Líneas Futuras

Las líneas de desarrollo futuro de este proyecto no son una lista genérica de tecnologías interesantes: emergen directamente de lo que las pruebas revelaron y de las limitaciones identificadas durante el proceso.

- **Piloto en entorno industrial real.** La visita a Mercafilo durante el Sprint 9 demostró que el problema que este proyecto resuelve existe y afecta a equipos comerciales reales. Sin embargo, el panel de validación estuvo formado por evaluadores universitarios en condiciones controladas. El siguiente paso es un piloto en producción con el equipo comercial de una PYME del sector, durante al menos un trimestre, para medir la adopción real, identificar el vocabulario técnico específico del sector y ajustar los prompts del sistema a ese dominio concreto. Esa sería la validación que convertiría el prototipo en producto.
- **Extensión al módulo de fabricación (MRP).** En Mercafilo se detectó que el registro manual no se limita al área comercial: las órdenes de fabricación, los consumos de material y los partes de trabajo presentan exactamente el mismo cuello de botella operativo. La arquitectura de inyección dinámica de esquema está diseñada precisamente para esto: añadir MRP no requiere reescribir ningún componente del sistema, sino validar los nuevos modelos de Odoo con el mecanismo ya operativo. Es la línea de mayor impacto industrial con menor coste de desarrollo incremental.
- **Resiliencia ante conectividad intermitente.** En una nave industrial o en campo abierto, la señal de datos puede ser débil o inexistente. Una extensión natural sería implementar una cola local en el dispositivo que almacene los registros pendientes cuando no hay conexión y los sincronice con Odoo en cuanto se restablece, manteniendo la experiencia de usuario sin interrupciones. Esta capacidad convertiría el bot en una herramienta confiable para entornos de trabajo exigentes.
- **Ampliación a WhatsApp Business API.** La mensajería predominante en el entorno empresarial español es WhatsApp, no Telegram. Dado que el bot está desacoplado del canal de mensajería la lógica de IA y el conector Odoo son independientes de la capa de entrada, adaptar la solución a WhatsApp Business API es una extensión de alto impacto con coste de ingeniería moderado. Ampliaría el alcance del sistema a empresas donde Telegram no es la plataforma habitual.
- **Bot bidireccional:** notificaciones proactivas. Actualmente el bot responde a peticiones del usuario. La siguiente evolución lógica es que el bot tome la iniciativa: alertar al comercial cuando le asignan un nuevo lead, notificar al responsable cuando el stock de un producto cae por debajo del umbral definido o recordar el seguimiento de una oportunidad con fecha próxima. Eso transformaría la herramienta de reactiva a proactiva, cerrando el ciclo de información entre el ERP y el personal de campo.

REFERENCIAS

- Adamopoulou, E., & Moussiades, L. (2020). Chatbots: History, technology, and applications. *Machine Learning with Applications*, 2, 100006. <https://doi.org/10.1016/j.mlwa.2020.100006>
- AHORA Soluciones. (2026). AHORA ERP: Software de Gestión Empresarial Freeware. <https://www.ahora.es/>
- Bengio, Y., Ducharme, R., Vincent, P., & Jauvin, C. (2003). A neural probabilistic language model. *Journal of Machine Learning Research*, 3, 1137-1155.
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., ... & Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877-1901.
- Chowdhury, G. G. (2003). Natural language processing. *Annual Review of Information Science and Technology*, 37(1), 51-89. <https://doi.org/10.1002/aris.1440370103>
- Dale, R. (2016). The return of the chatbots. *Natural Language Engineering*, 22(5), 811-817. <https://doi.org/10.1017/S1351324916000243>
- Davenport, T. H. (1998). Putting the enterprise into the enterprise system. *Harvard Business Review*, 76(4), 121-131.
- Davis, F. D. (1989). Perceived usefulness, perceived ease of use, and user acceptance of information technology. *MIS Quarterly*, 13(3), 319-340. <https://doi.org/10.2307/249008>
- Deloitte Insights. (2020). *Automation with intelligence: Pursuing organisation-wide reimagination*. Deloitte Development LLC. <https://www2.deloitte.com/us/en/insights/focus/technology-and-the-future-of-work/intelligent-automation-2020-survey-results.html>
- European Commission. (2021). *Proposal for a Regulation on Artificial Intelligence (AI Act)*. Publications Office of the European Union. <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX%3A52021PC0206>
- Gartner. (2023). Top 10 Strategic Technology Trends for 2023. Gartner Research. <https://www.gartner.com/en/articles/gartner-top-10-strategic-technology-trends-for-2023>

- Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* [Doctoral dissertation, University of California, Irvine].
- Google DeepMind. (2023). *Gemini: A Family of Highly Capable Multimodal Models*. Technical Report. https://storage.googleapis.com/deepmind-media/gemini/gemini_1_report.pdf
- Hammer, M., & Champy, J. (1993). *Reengineering the corporation: A manifesto for business revolution*. HarperBusiness.
- Infor. (2022). Infor Coleman AI Digital Assistant. Infor. <https://www.infor.com/products/coleman>
- Jurafsky, D., & Martin, J. H. (2024). *Speech and Language Processing* (3rd ed. draft). <https://web.stanford.edu/~jurafsky/slp3/>
- Kumar, K., & Hillegersberg, J. V. (2000). ERP experiences and evolution. *Communications of the ACM*, 43(4), 22-26. <https://doi.org/10.1145/332051.332063>
- Lasi, H., Fettke, P., Kemper, H. G., Feld, T., & Hoffmann, M. (2014). Industry 4.0. *Business & Information Systems Engineering*, 6(4), 239-242. <https://doi.org/10.1007/s12599-014-0334-4>
- Nambisan, S., Lyytinen, K., Majchrzak, A., & Song, M. (2017). Digital innovation management: Reinventing innovation in the digital world. *MIS Quarterly*, 41(1), 223-238. <https://doi.org/10.25300/MISQ/2017/41:1.03>
- Odoo S.A. (2024). *Odoo 17 Documentation: Web Service API*. https://www.odoo.com/documentation/17.0/developer/api/external_api.html
- Radziwill, N. M., & Benton, M. C. (2017). Evaluating quality of chatbots and intelligent conversational agents. *Software Quality Professional*, 19(3), 25-36.
- Richardson, L., & Ruby, S. (2007). *RESTful Web Services*. O'Reilly Media.
- Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
- SAP SE. (2023). SAP Announces Joule, a Natural-Language, Generative AI Copilot. SAP News Center. <https://news.sap.com/2023/09/joule-new-generative-ai-assistant/>
- United Nations. (2015). *Transforming our world: The 2030 Agenda for Sustainable Development* (Resolution A/RES/70/1). United Nations General Assembly. <https://sdgs.un.org/2030agenda>

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. In I. Guyon et al. (Eds.), *Advances in Neural Information Processing Systems 30* (pp. 5998-6008). Curran Associates.
- Venkatesh, V., Morris, M. G., Davis, G. B., & Davis, F. D. (2003). User acceptance of information technology: Toward a unified view. *MIS Quarterly*, 27(3), 425-478. <https://doi.org/10.2307/30036540>
- Weizenbaum, J. (1966). ELIZA—A computer program for the study of natural language communication between man and machine. *Communications of the ACM*, 9(1), 36-45. <https://doi.org/10.1145/365628.365636>
- Winer, D. (1999). XML-RPC Specification. UserLand Software. <http://xmlrpc.com/spec.md>
- Zhang, Z., Geiger, J., Pohjalainen, J., Mousa, A. E., Jin, W., & Schuller, B. (2018). Deep learning for environmentally robust speech recognition: An overview of recent developments. *ACM Transactions on Intelligent Systems and Technology*, 9(5), 49. <https://doi.org/10.1145/3178115>

ANEXO I

El código fuente completo de este proyecto, junto con todos los archivos de configuración y la documentación adicional, se encuentra alojado en el siguiente repositorio de GitHub, cuyo acceso es obligatorio para la evaluación y revisión detallada del mismo:

[byronperez068p-collab/ODOO-VOZ: LLENAR EL ODOO CON LA VOZ](#)

ANEXO II

ANEXO - DECLARACIÓN DEL USO DE INTELIGENCIA ARTIFICIAL

En la elaboración del presente TFG se han utilizado herramientas de Inteligencia Artificial (IA) como apoyo en distintas fases del trabajo académico.

Las herramientas utilizadas han sido ChatGPT, Gemini, Copilot, Claude, Perplexity, GitHub Copilot, *Otras (especificar)*.

Estas herramientas se han utilizado para las siguientes tareas:

- Búsqueda preliminar y síntesis de información.
- Generación y organización de ideas.
- Generación de esquemas o estructuras de contenido.
- Apoyo a la redacción y mejora de estilo académico.
- Traducción o reformulación de textos.
- Generación, revisión o depuración de código software.
- Generación de ejemplos, casos de uso o datos simulados.
- Apoyo en análisis de datos o interpretación de resultados.
- *Otras finalidades (especificar)*

DECLARO QUE:

- El contenido final presentado es resultado de mi trabajo personal y académico.
- He revisado y validado críticamente toda la información obtenida mediante herramientas de IA.
- El uso de IA no ha sustituido el análisis, la reflexión ni las conclusiones propias exigidas en el TFG.
- El trabajo cumple las normas de integridad académica y propiedad intelectual establecidas por la universidad.

Nombre del estudiante: Byron Andres Perez Chavez

Firma:

